



VoiceNet: A Deep Learning-Based Framework for Biometric Recognition Using Voice

Prepared by

Albatoul Aljuhani

Raneem Almousa

Joury Khalid

Supervised By

Dr. Rania Ghoniem

A Graduation Project Report Submitted to The College of Computer and Information Sciences at
PNU in Full Fulfillment of the Requirements for The Degree of Bachelor of Science in
Information Technology

8WS

CCIS, PNU

Riyadh, KSA

1447

2025-2026

Table Of Contents

Chapter 1: Introduction	9
1.1 Background & Motivation	9
1.2 Problem Statement.....	11
1.3 Objectives of the Project.....	12
1.3.1 Alignment with the UN Sustainable Development Goals (SDGs):	13
1.4 Scope and Limitations	14
1.4.1 Significance of the Project.....	15
Chapter 2: Background and Related Work	17
2.1 Studies Using Traditional (Pre-Deep Learning) Methods	17
2.2 Studies Based on Deep Learning Approaches	18
2.3 Comparative Analysis (Gap Analysis) and the Need for VoiceNet.....	19
Chapter 3: System Analysis	21
3.1 Requirement Specification	21
3.1.1 Functional Requirement	21
3.1.2 Non-Functional Requirement.....	22
3.2 Requirements Analysis	23
3.2.1 Requirement (Performance Requirement).....	23
3.2.2 Hardware Requirement (Implementation Requirement).....	24
3.3 System Flowchart.....	24
Chapter 4: System Design.....	25
4.1 System Architecture	25
4.2 The Model Training (Offline) Components & Processes	26
4.2.1 Data Acquisition (Public Dataset).....	26
4.2.2 Data Augmentation for Robustness.....	27
4.2.3 Feature Extraction (From 1D Audio to 2D Image).....	27
4.2.4 The CNN-VGG16 Model	28
4.2.5 Role of CNN-VGG16 within the VoiceNet System	29
4.3 The Verification Process	30
4.3.1 Verification Process Module.....	30
4.3.2 Training module dataset (for offline module).....	31

4.3.3	Feature Extraction (MFCC& spectrogram)	32
4.3.4	Comparing the features through CNN- VGG16:.....	33
4.3.5	decision logic (threshold evaluation).....	34
4.4	The User Enrollment (Live) Components & Processes	35
4.4.1	User Interaction	36
4.4.2	Enrollment Module	36
4.4.3	Capture and Feature Extraction	36
4.4.4	Secure Template Generation and Database Storage	37
4.4.5	Significance of the Enrollment Process	37
4.5	User Interface Design	38
Chapter 5:	Implementation.....	39
5.1	Implementation Requirements:.....	39
5.2	Implementation Details:	40
5.2.1	Environment and Toolbox Configuration	40
5.2.2	Preprocessing and Data Augmentation.....	41
5.2.3	Feature Extraction Implementation.....	41
5.2.4	CNN Model Training.....	42
5.2.5	Decision Logic and Thresholding	42
5.3	I/O Screens:.....	43
5.3.1	System Training.....	43
5.3.2	System Registration of live voice	46
5.3.3	: System Verification of identity live voice.....	49
Chapter 6:	Testing.....	50
6.1	Test plan:.....	50
6.2	Test cases:.....	52
6.3	Test results:	54
Chapter 7:	Conclusions	55
7.1	Evaluation:.....	55
7.2	Future work:	56
8. References		57
Appendix		58

List of Tables

Table 2-1 Shows the traditional methods used for voice authentication	17
Table 2-2 Comparison of the Proposed Model with existing studies.....	20
Table 6-1: The Test cases of (Login Attempts):	52

List of Figures

Figure 3-1 Flowchart Design of the System.	24
Figure 4-1 Block Diagram of the system components for the three main processes.	25
Figure 4-2 Sub block of the block diagram (Model training).	26
Figure 4-3 The 16 Layers of the VGG16 Model.	28
Figure 4-4 Sub-block of the Block Diagram (Verification Process).	30
Figure 4-5 This diagram shows the system's Threshold Evaluation.	34
Figure 4-6 Sub-block of the Block Diagram (User Enrollment).	35
Figure 4-7 Use Case Diagram shows how the user interacts with the system.	38
Figure 5-1 MFCC Visualization after the Spectrogram Generation.	41
Figure 5-2 MATLAB Training Progress for the VoiceNet VGG16 Model.	42
Figure 5-3: MATLAB Implementation for Data Loading and Spectrogram Preparation.	43
Figure 5-4: Signal Preprocessing and Spectrogram Feature Extraction.	44
Figure 5-5: Results of training in 10 Epoch to get a high accuracy rates with low loss.	46
Figure 5-6: MATLAB Script for Live User Registration and Reference Embedding Database.	46
Figure 5-7: Prepare the recorder to get the best voice capturing for them.	47
Figure 5-8: Lastly when the system take the four trails correctly a secure templet will be saved.	47
Figure 5-9: The output of the “Register” Script shows how the system checks the voice and builds a secure templet from 4 trials.	48
Figure 5-10: Live Verification Module, which will work with “Front-end” logic used during real-time Login attempts.	49
Figure 6-1: The results show the system meets our target calculations at 0.92 threshold.	50
Figure 6-2: The initial stage of testing is retrieving the scores we got from the trained data set.	50
Figure 6-3: Applying the calculations as discussed before and compare the results to the target score.	51
Figure 6-4: To show the performance visualization and EER calculation, used to determine the optimal balance between system security and user accessibility.	51
Figure 6-5: The visual curves to show the result of testing the system FAR and FRR.	54
Figure 0-1 VoxCeleb dataset.	58
Figure 0-2 LibriSpeech dataset.	58
Figure 0-3 TIMIT dataset.	59
Appendix B- The following table shows the shapes used for the elements in the Figure 0-4 Use Case Diagram:	59

List of Symbols and Abbreviations

Abbreviation	Meaning
PNU	Princess Norah University
CCIS	College of Computer and Information Sciences
GP1	Graduation Project 1
MFCCs	Mel-Frequency Cepstral Coefficients
CNNs	Convolutional Neural Networks
EER	Equal Error Rate
ATM	Automated Teller Machine
ID	Identity
SDGs	Sustainable Development Goals
ASR	Automatic Speech Recognition
SME	Small and medium enterprises
IDS	Identity system
AI	Artificial intelligence
UN	United Nations
FAR	False Acceptance Rate
FRR	False Rejection Rate
GMMs	Gaussian Mixture Models
UBM	Universal Background Model
EM	Expectation-Maximization
HMM	Hidden Markov Model
RNNs	Recurrent Neural Networks
LSTM	Long Short-Term Memory
GPU	Graphics Processing Unit
VGG16	Visual Geometry Group 16-layer model
TP	True Positive
FN	False Negative
FP	False Positive
TN	True Negative
FTE	Failure to Enroll
FTC	Failure to Capture
MATLAB	Matrix Laboratory

Acknowledgments

Alhamdulillah for making this journey possible and helping us to reach our goals and making it till this point to complete GP2. And may Allah help us to make our university proud of the graduates and help us reach our goals. Our best gratitude goes to our university (Princess Nourah bint Abdulrahman University) for always working to prepare the students professionally, academically, and personally, especially the College of Computer and Information Science, the faculty, and the IT department. We also sincerely thank our Supervisor, Dr. Rania Ghoneim, for her constant support, and we truly appreciate her efforts. Alhamdulillah for what we have achieved. We are grateful for all the support throughout this journey, and may Allah make the following steps filled with success.

Abstract

Biometric recognition using voice is a non-intrusive and natural approach to identity verification, with growing applications in security systems, mobile authentication, and access control. This project introduces VoiceNet, a deep learning-based framework designed for accurate and robust biometric recognition using voice signals. The system utilizes humans' voices as input and transforms them into spectrograms or Mel-frequency cepstral coefficients (MFCCs) to extract meaningful acoustic features. These features are then processed by a convolutional neural network (CNN) to learn speaker-specific patterns. To enhance performance and generalization, data augmentation techniques such as noise injection, pitch shifting, and time stretching are applied during preprocessing. The model will be trained and evaluated on publicly available voice biometric datasets, and its performance will be measured using accuracy, precision, recall, and equal error rate (EER). Experimental results aim to investigate whether VoiceNet achieves high recognition accuracy and resilience to variations in noise, accent, and speaking style.

Chapter 1. Introduction

1.1 Background & Motivation

The process of identifying or validating a person using their distinct physiological or behavioral traits is known as biometric recognition. Because of its dependability and difficulty of replication, it has become worthy in the fields of security, access control, and personal identification. Voice, iris, face, and fingerprint identification are commonly used, each with unique features and uses: [1]

- Fingerprint recognition: One of the first biometric techniques. Because of its great security, it has been routinely used in buildings, airports, and law enforcement.
- Facial recognition: This technique examines the distances between important facial features, and it is highly secure because it can even separate twins. It is commonly used by ATMs, social media, and mobile devices.
- Iris recognition: Provides great accuracy by utilizing the complex patterns of the iris. It is frequently found in high-security settings, including government ID programs, but it's rarely used due to the need for specialized scanners.

Although these methods are effective, their flexibility is sometimes limited by the need for either specialized devices or physical interaction. This has triggered the studies into more precise, practical, and easily integrated biometrics. We chose VoiceNet because voice-based, non-contact authentication represents the future of secure and inclusive technology. Unlike traditional methods, it allows people to access devices and services effortlessly. This project not only fixes real-world challenges like raising security only, but it also helps people with hand or finger mobility impairments who struggle to type passwords, use keypads, or interact with touchscreens. Voice-based authentication solves this problem by allowing them to access devices and services hands-free, making security both practical and inclusive. So, by focusing on voice recognition, we aim to create a solution that is practical, accessible, and helpful in modern digital security.

Several benefits have made voice-based biometric recognition an attractive choice:

- Natural: Voice recording is possible from a distance without physical contact, providing a useful experience.
- Easy acquisition: Unlike fingerprints or iris scans, Voice recognition does not require complex and specific devices.
- Flexible integration: Digital platforms, smart apps, telecommunications networks, and mobile devices can all use voice recognition.

The history of voice recognition technology is extensive. Simple pattern matching was the focus of early systems in the 1960s, which could barely detect the person's identity. Accuracy and scalability have increased because of research into increasingly sophisticated statistical models and machine learning approaches throughout the years. To accommodate differences in tones and background noise. Voice recognition is used today in a variety of fields:

- Banking and finance: Safe authentication over the phone and online.
- Mobile devices: Authorizing mobile apps and unlocking cellphones.
- Security and access control: allowing access to smart buildings, essential infrastructure, and restricted locations.
- Healthcare and e-learning: Identity verification for online learning environments and telemedicine consultations.

Voice-based systems have drawbacks despite their advantages, including background noise, mood swings, and shifting speech patterns. These elements have the potential to decrease accuracy, which encourages the creation of strong deep learning frameworks that can recognize speaker-specific patterns and adjust to changing circumstances. This project intends to improve security, usability, and accessibility by emphasizing voice recognition, demonstrating the usefulness of voice-based biometrics in a variety of applications.

1.2 Problem Statement

Biometric authentication has become an essential component of modern security systems, providing a convenient and reliable alternative to traditional password-based methods. Among various biometric modalities, voice authentication is gaining significant importance due to its non-contact and user-friendly nature.

However, voice biometric systems face several challenges that affect their accuracy and reliability. A person's voice can vary due to illness, aging, emotional state, or environmental conditions, making consistent identification difficult. Moreover, many existing systems rely on static pattern-matching techniques, which depend on fixed stored voice patterns and often fail to adapt to real-world variations in speech input.

To overcome these limitations, this project proposes VoiceNet, a deep learning-based voice biometric authentication system. VoiceNet employs a speaker detection framework that extracts unique speaker-specific features by transforming speech inputs into spectrograms and Mel-Frequency Cepstral Coefficients (MFCCs), which are then processed using Convolutional Neural Networks (CNNs). The model further enhances robustness through data augmentation techniques such as pitch fluctuation, time stretching, and background noise addition.

The system's performance will be evaluated using accuracy, precision, recall, F-score, Equal Error Rate (EER), and False Acceptance/False Rejection rates. Additionally, VoiceNet will be tested under real-world conditions, including accent variations, different tones, noisy environments, and varied speaking styles, to ensure its effectiveness and adaptability.

So, VoiceNet aims to advance secure authentication, smart environments, and access management by providing a more accurate and stronger voice-based biometric solution.

1.3 Objectives of the Project

Our project VoiceNet seeks to provide a secure, efficient, and user-friendly authentication framework that supports the strategic objectives of Saudi Arabia's Vision 2030 in digital transformation, cybersecurity, and smart city innovation.

General Objective:

To design and implement VoiceNet, a robust voice-based biometric recognition system.

Specific Objectives:

- Data Augmentation & Preprocessing:

Establish a comprehensive preprocessing pipeline that employs data augmentation techniques (noise injection, pitch shifting, time stretching, and environmental simulations). This will enhance dataset diversity and improve the system's ability to generalize across real-world scenarios.

- Advanced Feature Extraction:

Develop and implement sophisticated algorithms, such as Mel-Frequency Cepstral Coefficients (MFCCs) and spectrogram analysis, to extract detailed acoustic and temporal characteristics of the human voice while maintaining stability under varying environmental conditions.

- Deep Learning Framework:

Design and train a Convolutional Neural Network (CNN) specifically tailored for speaker recognition. Explore the inclusion of advanced architecture.

- Evaluation & Validation:

The system will be evaluated using publicly available datasets (VoxCeleb, LibriSpeech, TIMIT). And evaluate performance using metrics such as accuracy, precision, recall, F-score, Equal Error Rate (EER), and False Acceptance/False Rejection Rates, ensuring VoiceNet meets global standards for biometric security.

- **Environmental Robustness:**

Strengthening the system's resilience against challenges such as background noise, quality check, and changes in speaking style, enabling reliable deployment in environments such as call centers, security checkpoints, and self-service kiosks.

- **Inclusivity & Accessibility:**

Promote equitable access by offering a dependable biometric solution for individuals with disabilities or users unable to utilize fingerprint or facial recognition systems, ensuring inclusivity in secure services and digital platforms.

- **Integration with Smart Cities and E-Government:**

Support smart city initiatives (e.g., NEOM, The Line) and e-government platforms by enabling seamless and secure voice authentication for banking, healthcare, transportation, and industrial applications, fostering safer and more efficient urban environments.

- **Contribution to Vision 2030 and AI Research:**

Advance research and innovation in AI-powered biometric security, contributing to cybersecurity, digital innovation, and the knowledge-based economy. This also fosters the development of local expertise and positions Saudi Arabia as a leader in voice recognition technologies.

1.3.1 Alignment with the UN Sustainable Development Goals (SDGs):

- **SDG 4: Quality Education (Indirect):** Enables secure access to e-learning platforms through voice-based authentication, ensuring only authorized users can participate.
- **SDG 9: Industry, Innovation, and Infrastructure** – through the development of advanced voice biometric technologies.
- **SDG 10: Reduced Inequalities** – by creating accessible and inclusive verification solutions for diverse populations.
- **SDG 16: Peace, Justice, and Strong Institutions** – by ensuring trustworthy authentication mechanisms that reduce fraud and identity theft.

1.4 Scope and Limitations

The endeavor is to use voice biometric analysis as more than a mere pitch, tone, and frequency-based verification tool, but to integrate it into an encompassing authentication system that also uses the underlying features. For instance, how fast or slow it takes an individual to put words into speech, often referred to as the ‘speech-rate’, can be a subtle yet distinct signature. Likewise, the rhythm of inhalation pauses taken during speaking is generally stable and can offer another level of verification.

The technology applies to a variety of industries. Inside airports and at border control, it could cut down on lines by confirming passengers’ identities faster than manual checks. Voice biometrics means using one’s own voice to control the voice assistants – Siri, Alexa, and Google Home – to reduce wrongful usage. e-Government applications could also benefit from such technology and help to ensure secure service of elderly and handicapped people, who have difficulties entering a password or carrying a physical token. Also, sellers on e-commerce platforms might deploy voice verification as an additional security feature when verifying the buyers’ identities before completing the transactions.

The initiative is also looking at combining voice biometrics with other cutting-edge technologies. When added to blockchain, voiceprints could be held securely in tamper-resistant ledgers. Moreover, the implementation of artificial intelligence-based noise reduction can help the system differentiate between voices with high accuracy, even within a crowd or a noisy environment, thus applicable.

To maintain a clear focus, the VoiceNet project has some boundaries:

- **Speaker Verification Only:** This project focuses on speaker verification (confirming a speaker's identity). It is not designed to be an Automatic Speech Recognition (ASR) system, so it will not understand or convert speech to text.
- **Exclusion of Emotional Analysis:** While mentioned as an area of interest, the analysis of emotional states (such as stress or fatigue) is not included in the objectives of this project. The system's sole purpose is identity authentication.

1.4.1 Significance of the Project

- Practical Significance

Voice biometrics provides a fundamental shift in authentication with the potential to implement an identity system based on single sign-on. Instead of serially recollecting all your numerous passwords, all your different platforms could be unlocked by speaking with your individual voiceprint. This is not just convenient, it's also a far more secure method of accessing. The IDS can also be quite attractive to small and medium enterprise (SME) businesses without a lot of money to spend on expensive security hardware. Voiceprint verification provides affordable security for many of these companies, allowing them to secure sensitive information and online transactions. Yet another real-world advantage is enabling digital voice signatures that can legally bind important transactions in a secure way. In banking, healthcare, and online contracts, a voiceprint could become an acceptable replacement for scribbled or electronic signatures, with the potential to cut fraud.

- Alignment with Saudi Vision 2030

This initiative is in line with the Saudi Vision 2030, where Digital transformation, innovation, and cybersecurity are some of the main objectives set forth. By logging in without a password, possible from platforms like Absher and Tawakkalna, voice authentication improves e-government productivity. It is also a key component of smart city projects like NEOM and The Line, where smart devices, homes, and vehicles are anticipated to seamlessly integrate with the humans that utilize them. In these surroundings, voice verification is required to ensure that only permitted users can use the networked technologies. On a macro level, voice biometrics has led to the development of AI and cybersecurity startups in Saudi Arabia that create an ecosystem supporting entrepreneurship and knowledge-based industries. It also minimizes the use of expensive, resource-draining physical identification systems while facilitating sustainable innovation.

- Relevance to UN Sustainable Development Goals (SDGs)

UN SDGs with which voice biometrics has a direct relationship. It contributes to SDG 10 (Reduced Inequalities) by offering universal access to digital systems for people with disabilities and leaving no one behind in the digital age. It falls under SDG 11 (Sustainable Cities and Communities), and it backs the safe incorporation of smart systems into urban operations from clinics to public transportation networks. In addition, the technology is compatible with SDG 9 (Industry, Innovation and Infrastructure) by supporting advanced and resilient digital infrastructure that enables innovation in the long term.

- People with Disabilities

This project promotes inclusivity by offering voice-based authentication that enables people with disabilities to access digital services easily and securely. It supports the blind people, the motor-disabled, and the elderly by removing the need for typing or remembering passwords.

Chapter 2. Background and Related Work

As we mentioned in Section 1.1 Background & Motivation, our motive to choose the Voice Recognition System instead of other Biometric systems was because the Voice system is a voice-based, non-contact authentication system that represents the future of secure and inclusive technology. Unlike traditional methods, it allows people to access devices and services effortlessly.

2.1 Studies Using Traditional (Pre-Deep Learning) Methods

Back in the day, before deep learning technology was established, Traditional statistical Deep Learning was the start for voice authentication systems to verify a person's identity by their voice. It involved extracting their features using Mel-Frequency Cepstral Coefficients (MFCCs), and it's still used because it's the standard feature extraction in voice authentication systems. Here is a comparison of the old statistical models.

Table 2-1 Shows the traditional methods used for voice authentication

Comparison	Gaussian Mixture Models (GMMs)	GMM-Universal Background Model (GMM-UBM)	Hidden Markov Model (HMM)
Concept	By using a Gaussian Mixture, each Gaussian will represent the pattern of the user's voice.	The GMM-UBM approach adapts a pre-trained UBM parameter to create speaker-specific models to target the identity.	A statistical model is represented as a sequence of hidden states.
Main use	Speaker modeling	Speaker verification	Speech recognition
Strengths	Simple and effective.	Faster and more accurate than GMMs.	Perfect for timedependent data.
Limitations	Weak and needs a large dataset.	Weak performance.	Complex training.

So, the traditional models were the start for speaker recognition and were effective at their time. But died because of its weakness and may get confused when two people have a little similarity in

their voices. And in our modern life, we need a stronger method to detect identity and avoid granting access to unauthorized people.

2.2 Studies Based on Deep Learning Approaches

Deep learning has recently changed speaker recognition in a big way. Old methods needed features to be designed by hand, but deep learning models can automatically learn complex patterns (like pitch, tone, and rhythm) straight from audio files or visual spectrograms. This change has made new models much more accurate and adaptable.

1) Convolutional Neural Networks (CNNs): Early deep learning studies, like the one by Nagrani (2017), which introduced the VoxCeleb dataset, used CNNs and had success. CNNs are great at handling grid-like data, so they are perfect for learning key features from spectrograms (visual images of audio).

- Weakness: The problem was that these models focused too much on "short sound pieces" and missed the wider context of the speech. Because of this, their accuracy dropped when there was "background noise".

2) Recurrent Neural Networks (RNNs) and (LSTMs): To understand longer-term patterns and context, researchers started using RNNs and LSTM models. This was an upgrade because these models can "remember" previous information, helping them understand long audio clips better.

- Weakness: The main issue here was practicality. These models are "heavy," which means they "take a lot of time and power to train". They also "overfit easily when the dataset is small," which makes them hard to use in the real world.

And here is why VoiceNet's Approach is going to be better. VoiceNet was created to avoid these issues by finding a balance between good performance and practical use.

- To fix the weakness of early CNNs: VoiceNet uses a lot of data augmentation. By adding things like noise, pitch changes, and time stretching, the model learns to deal with new voices and noisy environments that made other models fail.

- To fix the weakness of large (RNNs) and (LSTMs): VoiceNet focuses on being practical. It is built as an efficient CNN system that works in MATLAB. This makes sure it can run fast and reliably on regular computers without needing expensive hardware, which fits the project's hardware requirements.

2.3 Comparative Analysis (Gap Analysis) and the Need for VoiceNet

Recent advancements in speaker recognition have shifted from traditional statistical modeling to deep learning-based systems that allow for automatic and accurate voice feature extraction. The proposed system, VoiceNet, aims to create a strong deep learning framework for speaker recognition using Convolutional Neural Networks (CNNs) and Mel-Frequency Cepstral Coefficients (MFCCs), implemented in MATLAB.

Earlier studies [1] used Pre-Deep Learning methods like Gaussian Mixture Models (GMM) and Hidden Markov Models (HMM), which depended on manually designed acoustic features and statistical estimates. These systems performed well in controlled settings, but their accuracy dropped significantly in noisy or changing environments.

Later research [2] brought forward Deep Learning methods such as CNNs and RNNs, which automated feature extraction and enhanced accuracy. However, these approaches often needed large datasets and costly GPUs, and they were usually supported by Python-based frameworks.

More recent studies [3] have utilized Data Augmentation techniques to improve model robustness by enhancing training datasets with synthetic changes like pitch shifting, noise addition, and time stretching. These methods increased generalization across different conditions but still lacked efficient deployment processes in MATLAB environments.

Table 2-2 Comparison of the Proposed Model with existing studies.

Aspect	Proposed Model (VoiceNet)	Reference [1]	Reference [2]	Reference [3]
Problem Solved	Develops a robust speaker recognition system using CNN and MFCC in MATLAB for accurate and efficient authentication.	Performs speaker verification using handcrafted statistical models.	Automates voice feature extraction and enhances accuracy using neural networks.	Enhances robustness and generalization using data augmentation and normalization (pitch, spectral, WCMVN) to handle noise and device variability.
Classifiers / Techniques	CNN trained on MFCC and spectrogram features; deep learning implemented via MATLAB Deep Learning Toolbox.	GMM, HMM, and MFCC-based statistical modeling.	CNN, RNN, and LSTM architectures using TensorFlow or PyTorch.	Hybrid CNN–GMM models with cepstral normalization and multi-cepstral feature augmentation.
Software & Hardware	Implemented entirely in MATLAB using CPU/GPU capabilities; compatible with local lab environments.	MATLAB or C++ on CPU; minimal hardware requirements.	Python (TensorFlow/PyTorch) on high-end GPU setups.	Python (TensorFlow/PyTorch) with optional GPU support for real-time use.
Output / Results	High accuracy, reduced Equal Error Rate (EER), and stable performance in noisy and varied environments.	Moderate accuracy; degraded in noisy or accent-variable conditions.	High accuracy in benchmark datasets; reduced real-world adaptability.	Lower EER and higher accuracy under noisy and variable conditions.
Limitations	Requires dataset tuning and parameter optimization in MATLAB for larger datasets.	Sensitive to noise and channel variations.	Demands significant computational power and GPU training resources.	Higher training cost and need for parameter tuning.

Chapter 3. System Analysis

In voiceNet, the system will overcome the limitations by achieving these system requirements and will show how the user will interact with the system, as well as the flow of the system.

3.1 Requirement Specification

The VoiceNet system will perform a series of processes and tasks that are considered core processes to ensure that the VoiceNet system operates with high security and improved usability once these processes are done.

3.1.1 Functional Requirement

- Data Augmentation Functionality

The system should be capable of applying data augmentation methods, including pitch variation, time stretching, and background noise, in addition to the training dataset during the model development phase to enhance its robustness and performance.

- Enrollment Module

The system should allow the user to register their voice sample and their identity. After that, the system will capture this raw voice sample and assess the quality of that voice to ensure it is suitable for enrollment. After that, the system will process the raw voice to extract its unique features, the process is called (MFCCs) Mel-Frequency Cepstral Coefficients. Finally, the system will store the voice along with the identity (in a template).

- Recognition (Verification) Module

The system should offer an interface to allow the user to claim their identity, which is like starting by entering a username. After the claim, it must capture a live voice sample from the user (in a query). The captured voice sample will be evaluated to ensure it meets the quality requirements. Once verified, the system processes the query to extract key voiced features, including spectrograms and Mel-Frequency Cepstral Coefficients (MFCCs). To retrieve the identical stored 23 voice template from the database based on the claimed identity. Now the system will compare

the extracted features from the query with those of the stored template in a one-to-one matching process. And based on that, it will decide by producing a match score indicating how similar the two samples are, then evaluate the score against a predefined threshold to determine whether the claim is an authorized person or a fraud. Depending on the result, the system will either grant or deny the user access.

3.1.2 Non-Functional Requirement

3.1.2.1 Performance (Accuracy)

- False Accept Rate (FAR): Defines the system's security level. It is the probability of the system incorrectly matching an "impostor"(not matching the sample). The target FAR for VoiceNet should be less than or equal to 0.1%.
- False Reject Rate (FRR): Defines the system's convenience and usability. It is the probability of the system incorrectly rejecting a "genuine" user (matching the sample). The target FRR should be less than or equal to 12% at the specified 0.1% FAR.
- Equal Error Rate (EER): A key performance metric, representing the point where the FAR and FRR are equal, will be used for comparative analysis.

Also, some of the measures will be calculated after declaring the (TP, TN, FN, FP), such as:

- Accuracy: $\frac{TP+TN}{TP+FP+FN+TN}$
- Precision: $\frac{TP}{TP+FP}$
- Recall: $\frac{TP}{TP+FN}$
- F-Score: $2 \times \frac{Precision \times Recall}{Precision + Recall}$

3.1.2.2 Robustness

- The system must be resilient to high intra-user variations, which is the variability in the biometric feature set of a single individual. This includes maintaining robustness against physiological and emotional changes, such as voice alterations from medical conditions (e.g., a common cold) or emotional state. Furthermore, the system must maintain performance reliability despite changes in ambient conditions, specifically the presence of background noise. This is a primary objective of the VoiceNet framework.

3.1.2.3 Usability & Accessibility

- **Acceptability:** The system shall be highly acceptable to users, featuring a non-intrusive and convenient interface.
- **Failure to Enroll (FTE) Rate:** The system shall be designed to minimize the FTE rate, which is the proportion of users who cannot be successfully enrolled into the system.

3.1.2.4 Security

- **Data Confidentiality:** The system should prevent unauthorized access to sensitive user information, particularly the stored biometric templates (voiceprints).
- **Integrity:** The system should prevent the improper modification or destruction of stored data, ensuring the authenticity of the biometric templates.
- **Template Protection:** The system must ensure that stored biometric templates (voiceprints) are non-invertible.

3.1.2.5 Availability

- The system must guarantee timely and reliable access to and use of information for all legitimately authenticated users.

3.2 Requirements Analysis

Our system will depend on both Software (MATLAB) and hardware (PC), to establish its requirements, and here are the details:

3.2.1 Requirement (Performance Requirement)

We will use MATLAB R2025b to build and run the VoiceNet system. This MATLAB version has built-in support for the Deep Learning Toolbox, Signal Processing Toolbox, and Audio Toolbox, which we need for voice data preprocessing, feature extraction (MFCCs), and training the CNN model. With accuracy, precision, recall, F-score, Equal Error Rate (EER), and False Acceptance/False Rejection rates, this software environment makes sure that data is handled, models are tested, and performance is measured in a way that works well.

3.2.2 Hardware Requirement (Implementation Requirement)

To use the VoiceNet framework, all you need is a personal computer with an Intel Core i7 processor, with at least 50 GB of free space to store the dataset and trained models.

3.3 System Flowchart

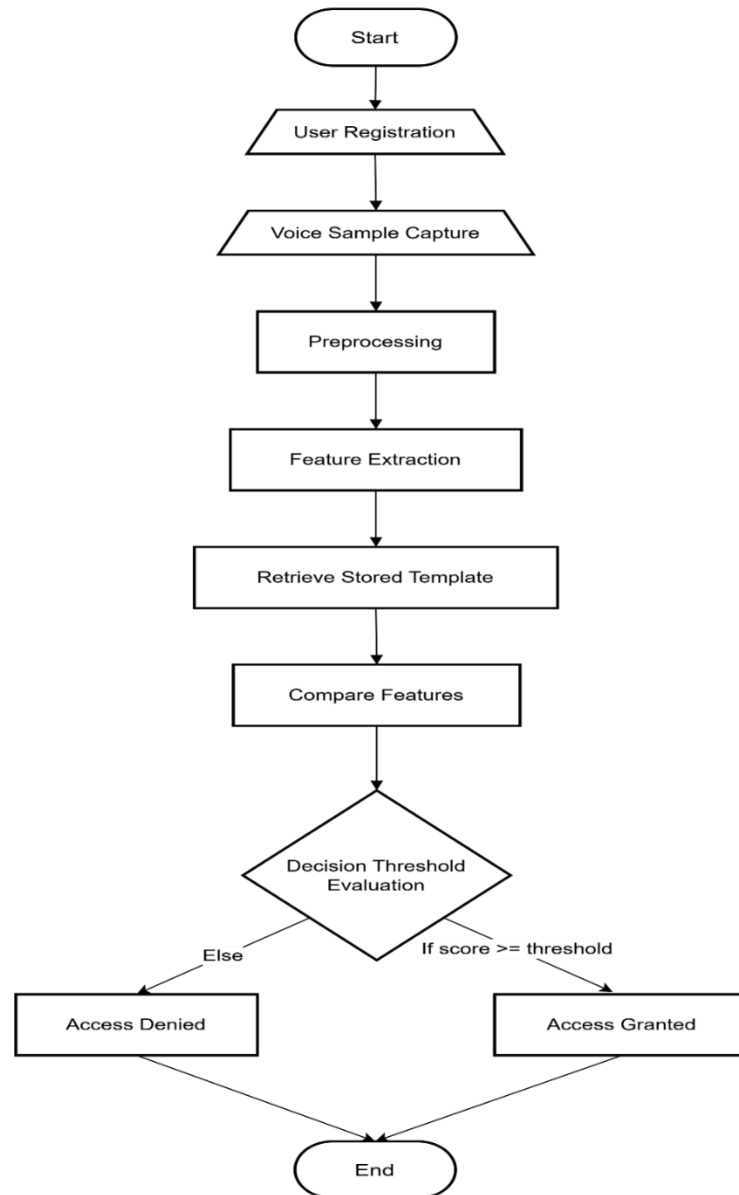


Figure 3-1 Flowchart Design of the System.

The following Flowchart shows how each event in the system will trigger the next till it reaches the threshold decision to determine whether to grant or deny access to the user.

Chapter 4. System Design

In this section, we will present the entire system's components (Offline Model Training, Verification Process, and Live User Enrollment) and how each event triggers the next.

4.1 System Architecture

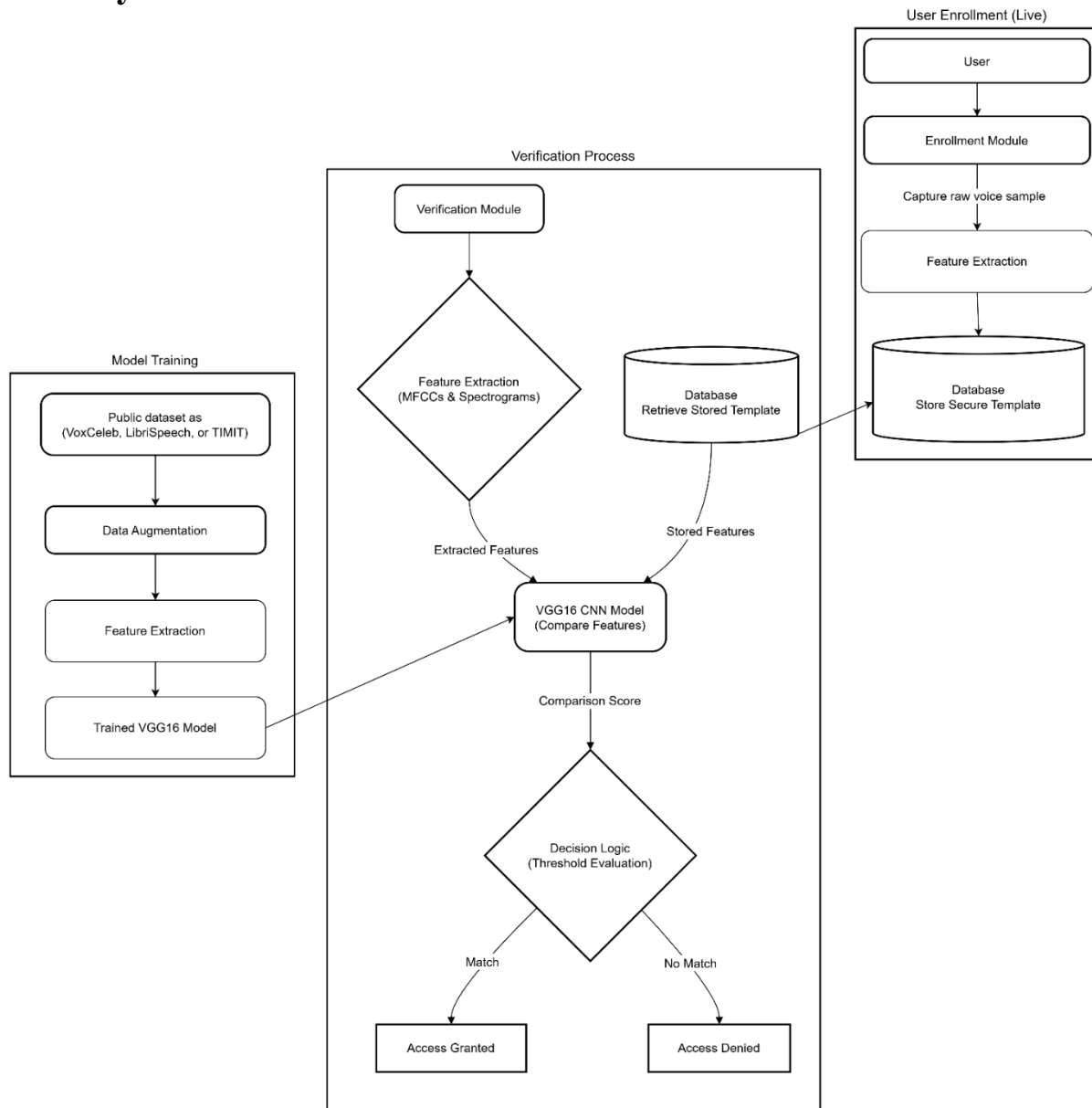


Figure 4-1 Block Diagram of the system components for the three main processes.

4.2 The Model Training (Offline) Components & Processes

Model training is the core of any "Deep Learning" system. This phase is critical for VoiceNet because it allows it to train a stronger model that can distinguish between different speakers easily. This process involves feeding the system large amounts of voice data in order to teach it the subtle patterns of human speech. The training process is divided into the following steps:

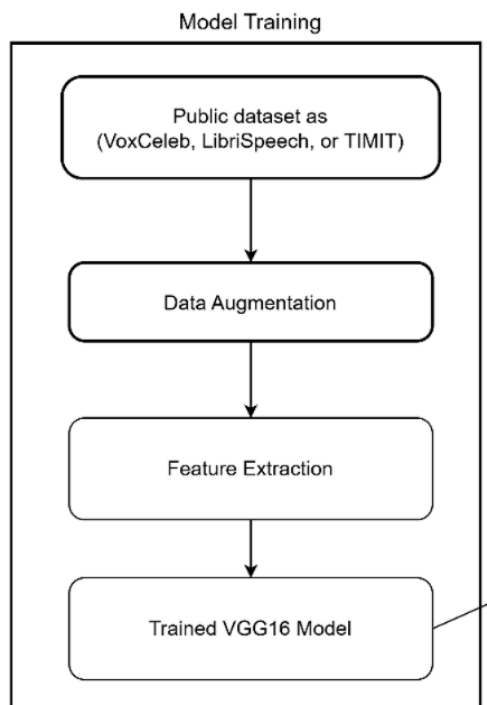


Figure 4-2 Sub block of the block diagram (Model training).

4.2.1 Data Acquisition (Public Dataset)

To train our VoiceNet system, we need a public dataset, and the data that we will be using is considered the foundation of the process. So, to properly train the VoiceNet system, we need a large and diverse public dataset. This task will be evaluated using standard benchmarks such as VoxCeleb, LibriSpeech, and TIMIT. These datasets are critical because they contain thousands of hours of speech from a diverse range of speakers, with varying accents, languages, and recording conditions. This diversity is essential for developing a generalized model that performs well outside of a laboratory setting.

4.2.2 Data Augmentation for Robustness

To improve the model's robustness, we will synthetically expand the training dataset via data augmentation. This process entails making modified copies of the original audio files to simulate real-world variability. As specified in our system requirements (3.1.1), this will include:

- **Background Noise Addition:** Introducing various types of ambient noise (e.g., street sounds, cafe chatter, office sounds) to train the model to focus on the speaker's voice and ignore distractions.
- **Pitch swings:** Adjusting the pitch of the speaker's voice to simulate natural variations in tone and rhythm.
- **Time stretching** is the process of increasing or decreasing the speed of an audio clip without changing the pitch, allowing for different speaking rates.

Training on this augmented data strengthens the model's flexibility to the challenges it will face in a live environment.

4.2.3 Feature Extraction (From 1D Audio to 2D Image)

A Convolutional Neural Network (CNN), such as VGG16, is intended to work with 2D data (such as images), because raw audio is a 1D waveform and is complex to work with. As a result, we must first convert the 1D audio signals into a 2D image that highlights speaker-specific features. This is the purpose of feature extraction. The main features of VoiceNet are:

- **Spectrograms:** visual representations of sounds. It plots frequency (Y-axis) versus time (X-axis), with color intensity representing the amplitude (loudness) of a specific frequency at a given moment. This effectively converts a sound clip into an image displaying its harmonic structure.
- **Mel-Frequency Cepstral Coefficients (MFCCs):** A common and highly effective feature in voice processing. MFCCs are a set of coefficients that collectively describe the short-term power spectrum of a sound. Crucially, they are calculated on the non-linear "mel scale," based on human auditory perception (how humans actually hear pitch). This makes MFCCs excellent at capturing a person's voice's unique timbre and formants (voice frequencies).

[2]

These 2D representations will be the "images" we feed into our CNN Model. The details of the Feature Extraction (MFCCs) and Spectrograms processes will be discussed in Section 4.3.

4.2.4 The CNN-VGG16 Model

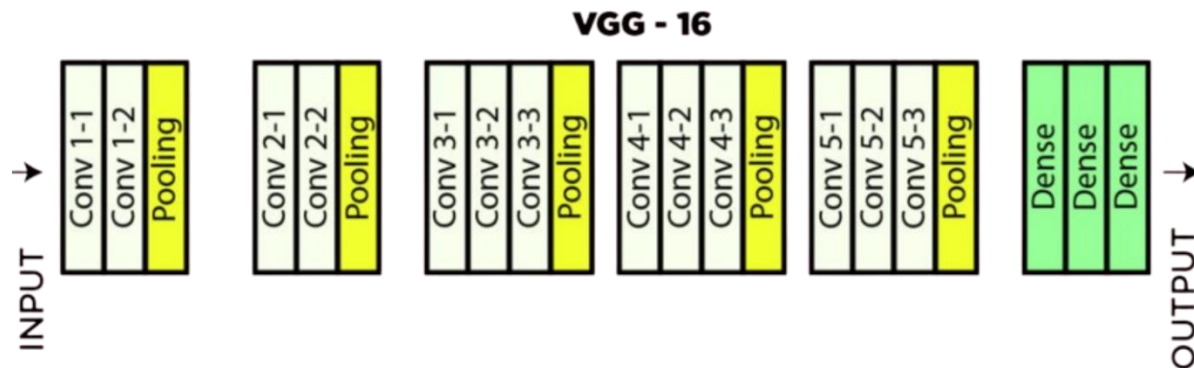


Figure 4-3 The 16 Layers of the VGG16 Model.

The 16 in VGG16 refers to 16 layers that have weights. The 16 layers that contain learnable parameters or weights are what give the VGG-16 model its name. The number 16 is derived from counting only the layers that are trained during the learning process, even though the complete architecture, as described in the diagram, has 21 distinct layers.

In particular, the model has 3 dense layers (which carry out the final classification) and 13 convolutional layers (which identify features). Because they lack weight to learn, the five Max Pooling layers, which are used to reduce dimensionality, are not included in this "16." As a result, the name is a combination of the three dense layers and the thirteen convolutional layers.

This is the final and most critical step of the training phase, which is the "mind" of where the model learns to understand the 2D voice features.

- What exactly is VGG16?

VGG16 is a 16-layer "deep" network with simple and effective architecture composed of stacked 3x3 convolutional filters. This design enables it to learn a diverse hierarchy of patterns, ranging from simple textures to complex shapes. The reason we chose

- the VGG16 model for VoiceNet?

While newer models of CNN are available, VGG16 was chosen for its optimal balance of high performance and practical implementation, which directly aligns with our project goals (as discussed in 2.2) of avoiding overly "heavy" or complicated models and hardware requirements, Section 3.2.2. [2]

4.2.5 Role of CNN-VGG16 within the VoiceNet System

The VGG16 model will not be trained as a simple classifier. Instead, it will be trained as a powerful embedding generator. (As shown in Figure 3):

- Training Objective: The model is trained to process a voice spectrogram and produce an embedding, which is a compact feature vector (such as a 1x128 array of numbers). The model is trained to generate similar vectors for various voice samples from the same person and very different vectors for samples from different people using sophisticated techniques like Triplet Loss.
- The Trained Model, the Final Product: This trained VGG16 model, which has learned to transform any voice spectrogram into a distinct 128-dimensional "fingerprint" or embedding, is the result of the entire offline phase.

As shown in Figure 4.3:

During enrollment, this model generates an embedding of the user's voice, which is saved in the database as their "Secure Template." And, during verification, this model generates a new embedding using the live voice sample. The "Comparison Score" is simply the mathematical distance (e.g., cosine similarity) between the new and stored template embeddings. This score determines whether the outcome is "Match" or "No Match".

4.3 The Verification Process

In this part of the system, it takes input from two sides: “the training data (offline)” and “the user enrollment (online)”. The (Verifying) process is the core process of the system and plays a significant role in it, and is the part where big decisions are made to ensure the system's security.

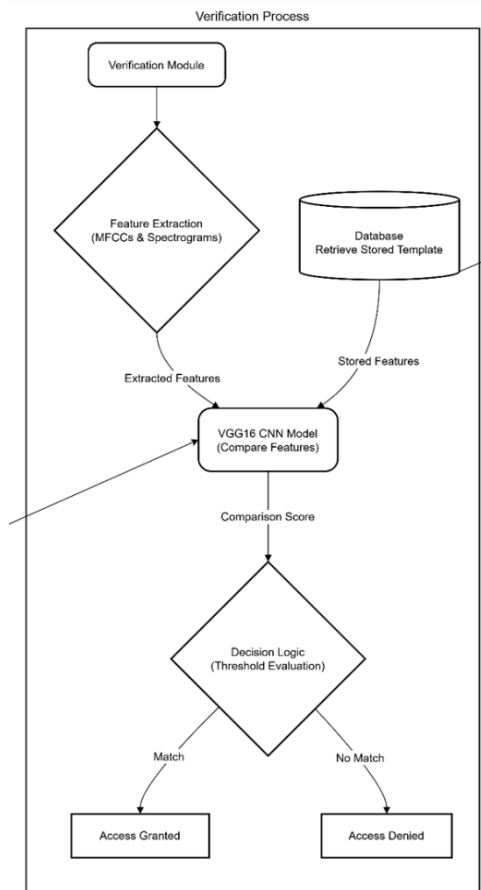


Figure 4-4 Sub-block of the Block Diagram (Verification Process).

4.3.1 Verification Process Module

The Verification Module serves as the system interface for processing login attempts, capturing username credentials, and recording new audio inputs for identity verification.

4.3.2 Training module dataset (for offline module)

The VoiceNet system depends on Model Training as its fundamental operation, which prepares and applies data for deep learning model training. The training process operates independently from user enrollment. The following essential steps explain the process:

4.3.2.1 Data Acquisition:

The VoiceNet system will be trained with public benchmark datasets, including VoxCeleb, LibriSpeech, and TIMIT.

These datasets are popular due to the variety of speech recordings from many speakers, featuring diverse voice rhythms and various recording environments.

4.3.2.2 Data Augmentation for Robustness:

The system generates artificial training examples to enhance model performance against actual system operating conditions.

The system generates new audio versions through three different methods, which include:

- **Background Noise Addition:** The system adds background noises to audio files to help the model identify the speaker's voice when background sounds are present.
- **Pitch Swings:** The system modifies speaker voice pitches to create authentic speech rhythm and tone variations.
- **Time Stretching:** The system changes audio speed without affecting pitch to help the model learn across different speaking speeds.

4.3.2.3 Feature Extraction:

Since the VGG16 CNN model is designed for 2D data (images), the raw 1D audio signals must be converted into a 2D image format that highlights speaker-specific characteristics.

The two main features extracted for the training model are: Spectrograms and Mel-Frequency Cepstral Coefficients (MFCCs).

4.3.3 Feature Extraction (MFCC& spectrogram)

The process of feature extraction converts user voice data into a format that is understandable by deep learning models.

The system performs two essential operations after voice sample collection and quality assessment: it converts the 1D audio waveform into a 2D representation. The VGG16 CNN Model requires 2D input data because it operates on images only.

The human ear responds to sound through Mel-Frequency Cepstral Coefficients (MFCCs), which extract voice characteristics from audio signals. The mel scale used for MFCC calculation produces coefficients that effectively represent human voice characteristics through timbre and formant frequencies.

The spectrogram displays the sound frequency distribution across time through a color-based representation of amplitude. The spectrogram shows the harmonic structure of the sound file by transforming it into an image format.

The combination of these features produces a distinctive voiceprint. The feature extraction module applies the trained VGG16 Model to analyze 2D features, which produces a fixed-size vector called the embedding that contains speaker voice characteristics. The system stores this embedding as the Secure Template.

The combination of MFCCs and Spectrograms enables the complete capture of both frequency and temporal voice characteristics.

4.3.4 Comparing the features through CNN- VGG16:

The Trained VGG16 Model performs two operations during live verification as described in 4.2.5. The model produces an embedding from the live sample features (Extracted Features) while it computes the Comparison Score through embedding distance calculation.

The VGG16 model conducts essential comparison operations during live verification.

- The model converts the user's live voice sample into a new embedding through feature extraction.
- The system determines the Comparison Score by comparing the two embeddings. The system determines the Comparison Score by calculating the distance between the live embedding and the stored template embedding from the database.

The Decision Logic receives the Comparison Score as input to determine whether to return a "Match" or "No Match" result.

4.3.5 decision logic (threshold evaluation)

As described in 3.1.2.1 Performance (Accuracy). The system uses this function to assess the comparison score against security threshold values. The system uses this threshold to determine if the two voice recordings match the claimed user's identity, deciding whether to grant or deny access.

The system grants access to users when their comparison score reaches or surpasses the established threshold. (TP-True Positive) or (FP-False Positive).

The system denies access to users when their comparison score does not meet the established threshold because it identifies them as impostors (TN-True Negative).

If the decision shows a (FN-False Negative) or (FP-False Positive), the user needs to add different samples of their voice to establish a higher score in (TP-True Positive) to grant access.

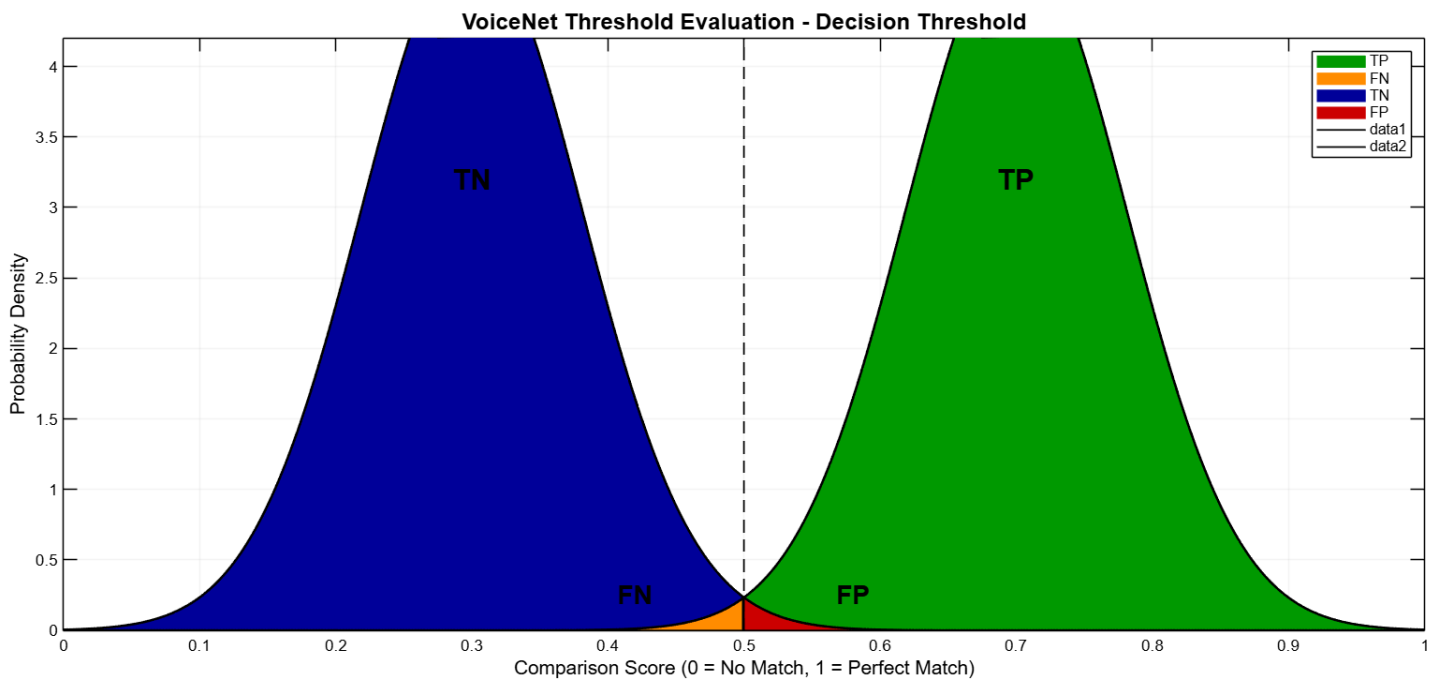


Figure 4-5 This diagram shows the system's Threshold Evaluation.

4.4 The User Enrollment (Live) Components & Processes

The user enrollment phase represents a crucial stage in the VoiceNet authentication lifecycle, as it establishes the foundation for accurate and secure speaker recognition during subsequent verification sessions [3] , [4] .This process occurs in a live environment, where a new user interacts directly with the system to create a unique biometric voice template [3]. The process consists of four primary components—User, Enrollment Module, Feature Extraction, and Database Storage—as illustrated in Figure 5: User Enrollment (Live) Block Diagram.

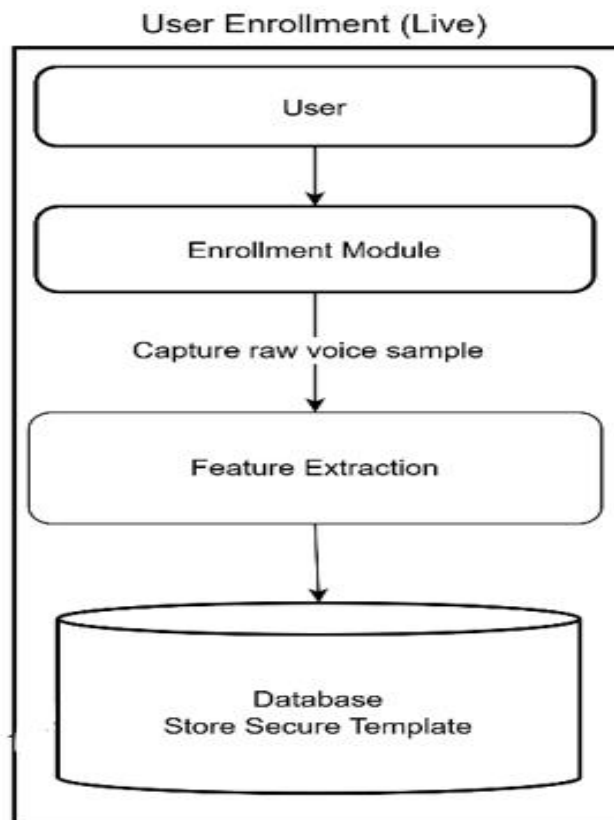


Figure 4-6 Sub-block of the Block Diagram (User Enrollment).

4.4.1 User Interaction

The first step in enrollment involves the user recording his voice input through the system interface. This action is usually done in a controlled setting to obtain high-quality input with reduced background noise [4]. The user is asked to record a predetermined phrase or sentence a number of times, giving the system enough voice variability to account for the natural fluctuations of tone, pitch, and rhythm [3] [5].

4.4.2 Enrollment Module

Once the raw voice input from the user is acquired, it then goes through the Enrollment Module, which executes several operations. The first thing it does is check the quality of the recording. In this context, it checks a few parameters, such as the signal-to-noise ratio, the level of clipping, as well as the duration [4]. If it detects any of the recordings below a minimum level of quality, it forces the user to record it all over again. The primary objective of this step is to only proceed with recordings of high quality [4].

4.4.3 Capture and Feature Extraction

Once the raw voice signal has been acquired successfully, the feature extraction step allows the 1D audio waveform to be transformed into a 2D representation, making it viable for deep learning processing. This is achieved via the processing of Mel Frequency Cepstral Coefficients

and spectrograms, thereby extracting both frequency as well as temporal features associated with the voice. [3]

- Spectrograms are a representation of the time domain signal in a frequency-time domain, plotting the intensity of the frequencies.
- MFCCs are a succinct, perceptually significant representation of the speech utterance, modeling the human auditory system's response to a sound.

All these features combined create a distinct “voiceprint” which embodies the personal speaker properties. The feature extraction module based on the CNN, as explained in Section 4.2, is

responsible for processing these features in order to obtain a fixed-size vector, a so-called embedding, expressing the speaker's voice. [3] [5]

4.4.4 Secure Template Generation and Database Storage

The generated embedding, also known as a secure voice template, is encrypted and retained within the system's database. This template becomes the model from which all future authentication attempts are derived. In order to preserve user privacy, the retained model within the system is rendered non-invertible, implying it can never be reversed to obtain the actual voice recording [3].

The database establishes a one-to-one relationship between the user's identity credentials, such as a username or user ID, and a voice template. The secure storage approach ensures data integrity and confidentiality, as specified under Section 3.1.2.4 [3].

4.4.5 Significance of the Enrollment Process

The entire VoiceNet system's accuracy and reliability are greatly influenced by this enrollment phase. The correct enrollment assures that every user's voice template is unique, of excellent quality, and securely kept. A properly performed enrollment process decreases Failure-to-Enroll (FTE) rates and increases recognition performance in actual verification situations. Furthermore, by integrating real-time feature extraction and strong storage methods, the live enrollment pipeline guarantees both security and user-friendliness, which are the project's main goals of accessibility, inclusivity, and digital security innovation [3] [4]

4.5 User Interface Design

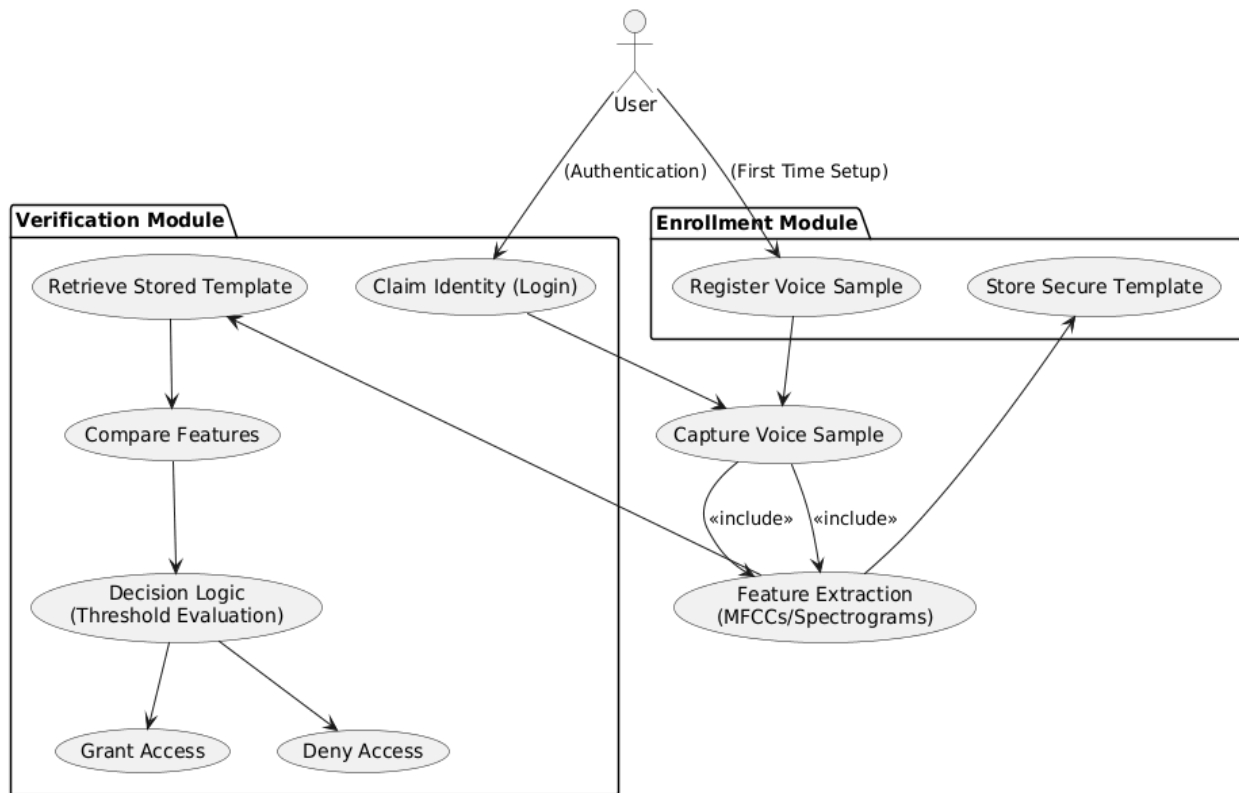


Figure 4-7 Use Case Diagram shows how the user interacts with the system.

The Use Case Diagram shows how the user interacts with the system in two ways: enrolling a new live sample (We will use the public access data sets), and the system will capture the features of the person's voice, then save it in a secure template. The next time the same user tries to enroll, the system will calculate the percentage of the matched claimed person's voice.

Chapter 5. Implementation

In this chapter we will discuss the system methodology that we will implement VoiceNet with such as what hardware and software will be used and what kind of data that is going to be used and how we are going to test the system inputs and by what methods.

5.1 Implementation Requirements:

As we discussed in GP1 we will build the VoiceNet system using a step-by-step plan. This plan starts with getting the data and ends with testing the final system. Following these steps will help make sure the system is strong, accurate, and meets all the project's goals. The plan is broken into these main phases:

- Tools and Equipment:

The whole system will be built using the latest version of MATLAB (R2025b). We will use MATLAB's Deep Learning Toolbox (for the CNN), Signal Processing Toolbox, Audio Toolbox (for preparing data and extracting features), and Statistics and machine learning Toolbox (essential for the Evaluation and Testing phase to calculate the biometric scores). For the hardware, we will use a personal computer (PC) that includes (CPU: Intel Core i7 / AMD Ryzen 7 or higher, RAM: ≥ 16 GB, GPU: NVIDIA CUDA-enabled GPU (RTX 2060, RTX 3060, or higher recommended, Storage: ≥ 500 GB SSD for datasets and model checkpoints).

- Getting and Preparing Data:

We will use large, free, public data sets like VoxCeleb, LibriSpeech, or TIMIT to train our model to be robust.

- The system will have two main parts:

Enrollment Module: This part lets a user sign up by recording their voice. The system saves the voice's features (MFCCs/spectrogram) as a secure template in a database.

Verification Module: This part handles logins. A user claims who they are and gives a new voice sample. The system pulls their saved template and compares the new sample to it to either approve or deny access.

- Extracting Features:

Raw voice recordings will be turned into useful sound-based formats. The main features we will extract are Mel-Frequency Cepstral Coefficients (MFCCs) and spectrograms. We chose these because they are good at capturing a speaker's unique voice qualities.

- Training Model:

The system will be trained on the data with a Convolutional Neural Network (CNN), specifically the VGG16 model. This deep learning model will be trained on the MFCCs and spectrograms. This teaches it to find the complex patterns needed to confirm someone's identity.

- Evaluation and Testing:

We will check the system's performance carefully using standard biometric scores, such as accuracy, precision, recall, F-score, Equal Error Rate (EER), and False Acceptance/False Rejection rates. We will also test how tough the system is in real-world situations, like in noisy places or when people speak in different styles or tones.

5.2 Implementation Details:

The VoiceNet System was fully implemented as a framework within the MATLAB R2025b environment by utilizing the high-performance toolboxes for processing, VGG16, and deep learning.

The following details outline the software, hardware and how it was integrated to achieve the project objectives:

5.2.1 Environment and Toolbox Configuration

The MATLAB environment is configured to integrate specialized toolboxes:

- Deep Learning Toolbox: The main toolbox to architect and train the VGG16 convolutional neural network layers.

- Audio and Signal Processing Toolboxes: Playing on the most critical point of the project by taking the input of raw audio signals and their subsequent transformation into visual feature representations (Spectrograms and MFCCs).
- Statistics and Machine Learning Toolbox: Used during the evaluation phase to calculate the biometric scores, including Accuracy, EER, FAR, and FRR.

5.2.2 Preprocessing and Data Augmentation

To enhance the robustness of the system, we developed a preprocessing script to expand the training datasets by using VoxCeleb, LibriSpeech, or TIMIT. The implementation included:

- Noise Injection: Adding background noises (street or cafe sounds) to train the model to ignore the noise and focus on the voice.
- Vocal Variations: Applying algorithms such as Time Stretching to simulate natural human rhythm and tone.

5.2.3 Feature Extraction Implementation

The core of the system involves transforming 1D raw audio signals into a 2D image suitable for the VGG16 model.

- Spectrogram: Implemented using Short-Time Fourier Transform (STFT) to map frequency density over time.
- MFCC: Extracted coefficients based on the non-linear Mel scale to simulate the human voice and capture unique timbre and formants.

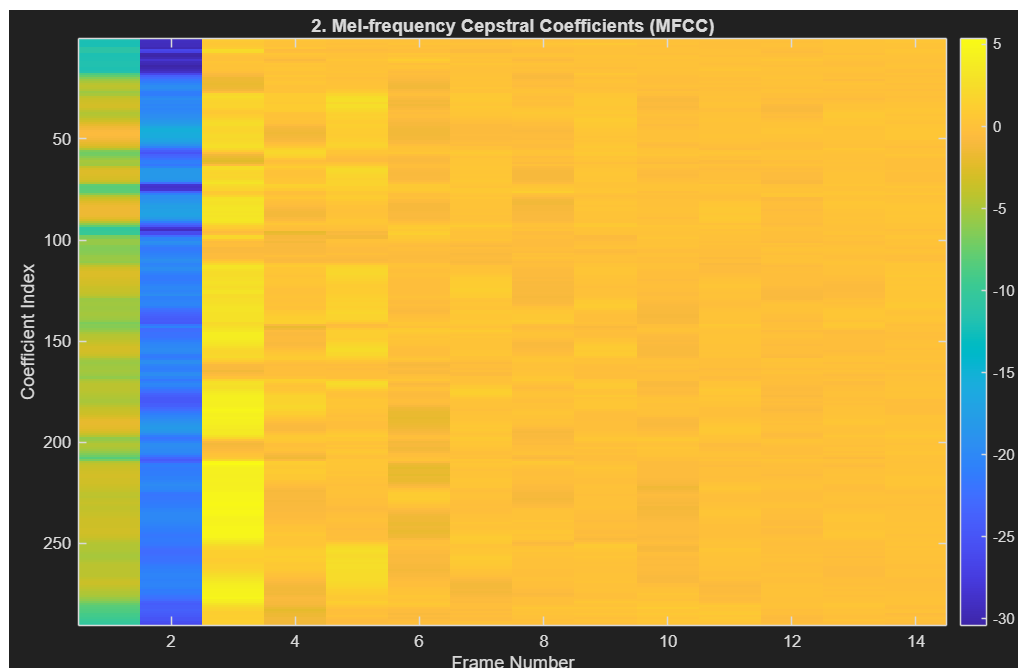


Figure 5-1 MFCC Visualization after the Spectrogram Generation

5.2.4 CNN Model Training

The VGG16 functions as a powerful embedding generator rather than a simple classifier.

- The model is trained to process a voice spectrogram to produce the voiceprint “128-dimensional feature vector.”
- The implementation is trained for so many voice prints to ensure that samples from the same person will produce similar vectors. On the other hand, samples from another person will produce different vectors that the system will notice and reject access.

5.2.5 Decision Logic and Thresholding

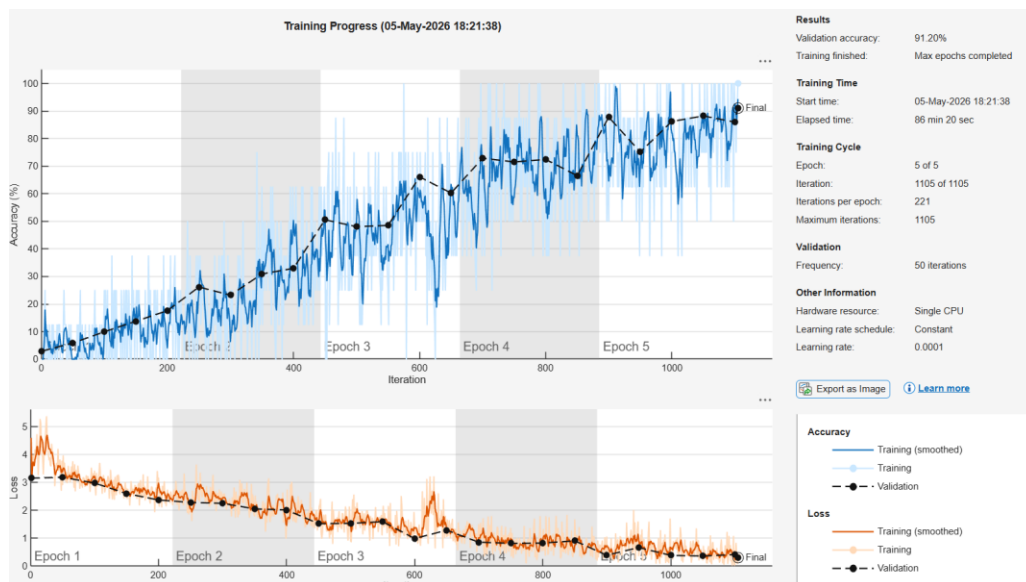


Figure 5-2 MATLAB Training Progress for the VoiceNet VGG16 Model

The verification process was performed to calculate the real-time mathematical distance between the live sample embedding and the Secure Template.

- The logical decision will compare the result of the Comparison Score against a predefined threshold to decide whether to grant access or deny it.
- If the score was equal to or more than the threshold, the system will push Access Granted to the claimed user; otherwise, Access Denied.

5.3 I/O Screens:

As we worked on the back end (Coding) & front end (GUI) of the system we will go through both described below:

5.3.1 System Training

The initial phase of the system is training the system on public data sets as mentioned before to make the system recognized and trained to distinguish each voice pattern with high level accuracy. So, it started with implementing the voice prints from the public datasets into the MATLAB project folder to be trained in the Main Script class that included the Loading Dataset & Preparing Spectrograms, Modifying VGG16 Architecture and VGG-16 training.

```
clc; clear; close all;
% 1. Loading Dataset & Preparing Spectrograms
disp('1. Loading Dataset & Preparing Spectrograms...');
scriptPath = fileparts(mfilename('fullpath'));
datasetPath = fullfile(scriptPath, 'train-clean-100');
if ~exist(datasetPath, 'dir')
    error('Error: Dataset directory not found at %s', datasetPath);
end
adsAll = audioDatastore(datasetPath, 'IncludeSubfolders', true, 'FileExtensions', {'.flac', '.wav'});
files = adsAll.Files;
speakerLabels = strings(numel(files), 1);
for i = 1:numel(files)
    parts = split(files{i}, filesep);
    speakerLabels(i) = string(parts(end-2)); % Fixed indexing for strings
end
all_cats = categorical(speakerLabels);
unique_speakers = categories(all_cats);
selected_speakers = unique_speakers(1:min(20, numel(unique_speakers)));
idx = ismember(speakerLabels, selected_speakers);
files = files(idx);
labels = removecats(all_cats(idx));
numFiles = numel(files);
X = zeros(224, 224, 3, numFiles, 'single');
fs_target = 16000;
for i = 1:numFiles
    [audioIn, fs] = audioread(files{i});
    if fs ~= fs_target
        audioIn = resample(audioIn, fs_target, fs);
    end
    audioIn = audioIn + 0.005 * randn(size(audioIn)); % Data Augmentation
    [S, ~, ~] = spectrogram(audioIn, hamming(1024), 512, 1024, fs_target);
    S_dB = 10 * log10(abs(S) + eps);
    specImg = imresize(mat2gray(S_dB), [224 224]);
    X(:, :, 1, i) = specImg; X(:, :, 2, i) = specImg; X(:, :, 3, i) = specImg;
end
```

Figure 5-3: MATLAB Implementation for Data Loading and Spectrogram Preparation.

This MATLAB script shows the initialization phase of the VoiceNet back-end, where the system establishes paths, validates the existence of the dataset, and prepares the audio datastore for feature extraction.

```

cv = cvpartition(labels, 'HoldOut', 0.2);
XTrain = X(:,:,cv.training); YTrain = labels(cv.training);
XTest = X(:,:,cv.test); YTest = labels(cv.test);

clear X;
disp('Memory cleared. Proceeding to training...');

% 2. Modifying VGG16 Architecture
net = vgg16;
lgraph = layerGraph(net);
numClasses = numel(categories(YTrain));
newFc = fullyConnectedLayer(numClasses, 'Name', 'new_fc', 'WeightLearnRateFactor', 10, 'BiasLearnRateFactor', 10);
newClassLayer = classificationLayer('Name', 'new_classoutput');
lgraph = replaceLayer(lgraph, 'fc8', newFc);
lgraph = replaceLayer(lgraph, 'output', newClassLayer);

% 3. Training VGG16
options = trainingOptions('sgdm', 'MiniBatchSize', 8, 'MaxEpochs', 5, 'InitialLearnRate', 1e-4, ...
    'Shuffle', 'every-epoch', 'ValidationData', {XTest, YTest}, 'Plots', 'training-progress');

trainedNet = trainNetwork(XTrain, YTrain, lgraph, options);

% 4. Save the Model and Test Data
save('VoiceNet_Model.mat', 'trainedNet', 'XTest', 'YTest');
disp('Success: Model and Test Data saved for reliability testing.');
```

Figure 5-4: Signal Preprocessing and Spectrogram Feature Extraction.

The previous figure shows the MATLAB implementation preprocessing pipeline, including noise-based data augmentation, 2D spectrogram generation, and dataset training for model validation. After that will create a tasting set before start training the system to save it for later.

Implementation of the Learning approach using the VGG16 architecture, showcasing the configuration of training hyper-parameters such as the SGDM (Stochastic Gradient Descent with Momentum) solver, mini-batch size, and learning rate schedules. To cut the loss and enhance the accuracy of results.

1. Loading Dataset & Preparing Spectrograms...

Memory cleared. Proceeding to training...

Training on single CPU.

Initializing input data normalization.

First the system will load the dataset and define the system data input and hardware to start processing the model training in 10 epoch.

The results of the training progress are fully displayed below...

=====								
Epoch	Iteration	Time Elapsed	Mini-batch	Validation	Mini-batch	Validation	Base Learning	
		(hh:mm:ss)	Accuracy	Accuracy	Loss	Loss	Rate	
=====								
1	1	00:00:48	0.00%	2.03%	3.8433	3.1600	1.0000e-04	
1	50	00:04:23	0.00%	5.87%	3.3708	3.1815	1.0000e-04	
1	100	00:07:53	12.50%	9.93%	3.1314	2.9454	1.0000e-04	
1	150	00:11:26	12.50%	8.80%	2.7430	2.7636	1.0000e-04	
1	200	00:15:10	12.50%	19.41%	2.7738	2.5524	1.0000e-04	

	2		250		00:18:43		0.00%		22.57%		2.8163		2.4527		1.0000e-04	
	2		300		00:22:12		25.00%		24.60%		1.9578		2.2932		1.0000e-04	
	2		350		00:25:42		12.50%		40.41%		2.1243		2.0941		1.0000e-04	
	2		400		00:29:14		25.00%		40.63%		1.9989		1.8217		1.0000e-04	
	3		450		00:32:45		37.50%		41.76%		2.2819		1.8468		1.0000e-04	
	3		500		00:36:17		25.00%		45.37%		1.8663		1.6526		1.0000e-04	
	3		550		00:39:47		25.00%		36.57%		3.2248		1.8608		1.0000e-04	
	3		600		00:43:16		25.00%		50.56%		1.7268		1.4349		1.0000e-04	
	3		650		00:46:47		37.50%		64.56%		1.2162		1.1354		1.0000e-04	
	4		700		00:50:18		75.00%		67.04%		1.2435		1.0823		1.0000e-04	
	4		750		00:53:49		87.50%		72.01%		0.8647		0.8503		1.0000e-04	
	4		800		00:57:23		87.50%		72.69%		0.5577		0.8369		1.0000e-04	
	4		850		01:00:59		50.00%		73.14%		1.3443		0.7602		1.0000e-04	
	5		900		01:04:36		62.50%		77.20%		1.1356		0.6950		1.0000e-04	
	5		950		01:08:10		87.50%		73.59%		0.2683		0.7308		1.0000e-04	
	5		1000		01:11:41		87.50%		83.07%		0.2473		0.4976		1.0000e-04	
	5		1050		01:15:12		75.00%		85.10%		0.5172		0.4248		1.0000e-04	
	5		1100		01:18:44		87.50%		84.65%		0.3574		0.4407		1.0000e-04	
	6		1150		01:22:15		75.00%		87.36%		0.4402		0.4168		1.0000e-04	
	6		1200		01:25:47		62.50%		83.97%		0.7250		0.4451		1.0000e-04	
	6		1250		01:29:18		87.50%		89.62%		0.2879		0.3350		1.0000e-04	
	6		1300		01:32:49		87.50%		89.16%		0.6375		0.3524		1.0000e-04	
	7		1350		01:36:21		62.50%		87.58%		0.4973		0.3541		1.0000e-04	
	7		1400		01:39:53		50.00%		89.62%		0.8092		0.3525		1.0000e-04	
	7		1450		01:43:25		100.00%		88.94%		0.1093		0.3019		1.0000e-04	
	7		1500		01:46:57		100.00%		92.55%		0.1748		0.2610		1.0000e-04	
	8		1550		01:50:29		100.00%		90.52%		0.0538		0.2657		1.0000e-04	
	8		1600		01:53:59		100.00%		93.45%		0.0571		0.2037		1.0000e-04	
	8		1650		01:57:33		100.00%		86.23%		0.0474		0.3713		1.0000e-04	
	8		1700		02:01:05		87.50%		93.00%		0.2308		0.2218		1.0000e-04	
	8		1750		02:04:36		100.00%		93.23%		0.0153		0.2215		1.0000e-04	
	9		1800		02:08:09		87.50%		95.71%		0.1497		0.1477		1.0000e-04	
	9		1850		02:11:41		87.50%		94.58%		0.2260		0.1671		1.0000e-04	
	9		1900		02:15:11		87.50%		93.68%		0.3173		0.2185		1.0000e-04	
	9		1950		02:18:39		100.00%		95.26%		0.0754		0.1689		1.0000e-04	
	10		2000		02:22:05		100.00%		95.94%		0.1511		0.1356		1.0000e-04	
	10		2050		02:25:32		100.00%		96.16%		0.0864		0.1379		1.0000e-04	
	10		2100		02:29:00		87.50%		94.13%		0.3840		0.1757		1.0000e-04	
	10		2150		02:32:26		100.00%		88.49%		0.0229		0.3975		1.0000e-04	
	10		2200		02:35:53		100.00%		88.94%		0.1423		0.2878		1.0000e-04	
	10		2210		02:37:09		100.00%		92.55%		0.0328		0.2321		1.0000e-04	

=====

Training finished: Max epochs completed.Success: Model and Test Data saved for reliability testing.

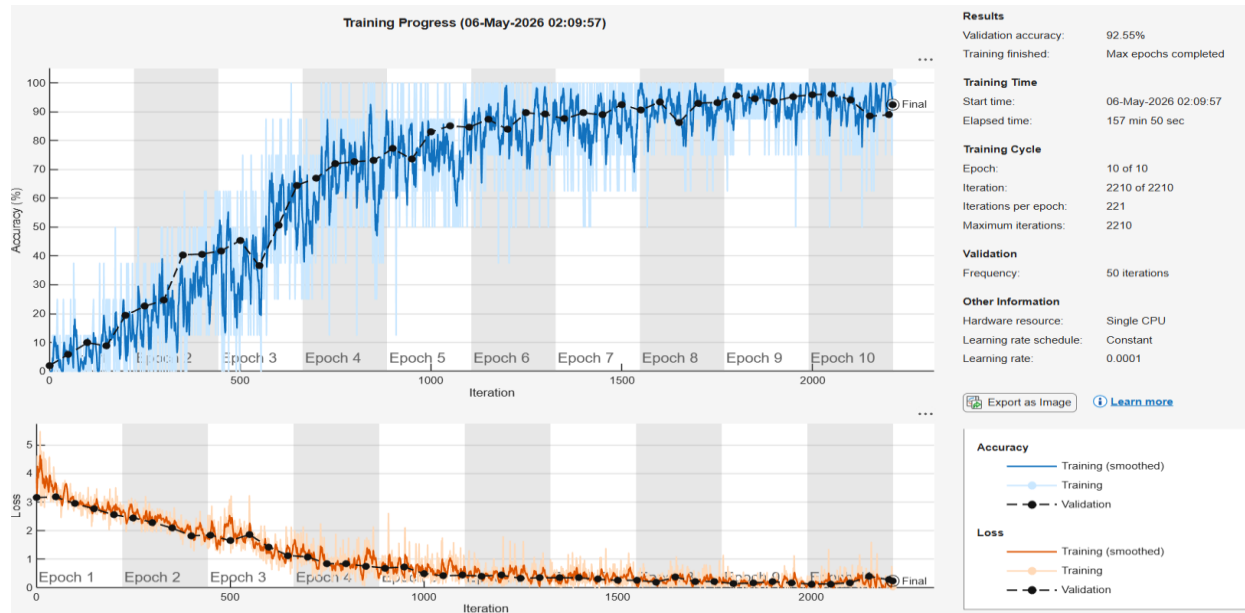


Figure 5-5: Results of training in 10 Epoch to get a high accuracy rates with low loss.

5.3.2 System Registration of live voice

The second Script shows how the system will add real users voice prints after training on the public datasets.

```

clc; clear; close all;
% 1. Load the VoiceNet Model
if exist('VoiceNet_Model.mat', 'file')
    load('VoiceNet_Model.mat', 'trainedNet');
else
    error('Error: Trained model not found. Run Main training first.');
```

```

end
userName = input('Enter Name for registration: ', 's');
% 2. Safe Database Loading
if exist('VoiceNet_Model.mat', 'file')
    m = matfile('VoiceNet_Model.mat');
    varList = who(m);
    if any(strcmp(varList, 'reference_embeddings'))
        load('VoiceNet_Model.mat', 'reference_embeddings');
    else
        reference_embeddings = containers.Map();
    end
else
    reference_embeddings = containers.Map();
end
% Check if user already exists
if isKey(reference_embeddings, userName)
    fprintf('\n[!] User "%s" already exists. Update voiceprint? (y/n): ', userName);
    if lower(input('', 's')) ~= 'y', return; end
end
fs_target = 16000; duration = 4; temp_embeddings = [];

```

Figure 5-6: MATLAB Script for Live User Registration and Reference Embedding Database.

The back-end logic for the live User Enrollment module, showing the initialization of the merging Inclusivity & Accessibility together. The user will be asked about their identity if they are registered the system will give them 4 seconds to prove that they are the claimed identity.

```

% 3. Voiceprint Registration with Quality Check
fprintf('\n Starting Voiceprint Registration for: %s \n', userName);

for i = 1:4
    validCapture = false; % Flag to ensure sound was heard

    while ~validCapture
        fprintf('\nTrial [%d of 4]: Ready? Press Enter and wait for "START"... \n', i);
        pause;
        % Prepare recorder
        recObj = audiorecorder(fs_target, 16, 1);
        disp('>>> [3... 2... 1...]');
        pause(1);
        disp('>>> START SPEAKING NOW! Say: Open VoiceNet system');

        recordblocking(recObj, duration);
        audioData = getaudiodata(recObj);
        % Calculate the peak volume to detect silence
        if max(abs(audioData)) < 0.02 % Increased threshold for better detection
            fprintf('\n[!] ERROR: No voice detected or sound too quiet. Please try again.\n');
            validCapture = false; % Keep the user in the while loop[cite: 1]
        else
            disp('>>> Recording completed. Processing...');
            validCapture = true; % Break the while loop and move to processing
        end
    end
end

% 4. Feature Extraction (Only for valid audio)
[S, ~, ~] = spectrogram(audioData, hamming(1024), 512, 1024, fs_target);
S_dB = 10 * log10(abs(S) + eps);
specImg = imresize(mat2gray(S_dB), [224 224]);
specImg_3D = cat(3, specImg, specImg, specImg);
% Generate Embedding using VGG16
emb = activations(trainedNet, specImg_3D, 'fc7', 'OutputAs', 'rows');
temp_embeddings = [temp_embeddings; emb];
end

```

Figure 5-7: Prepare the recorder to get the best voice capturing for them.

```

% 5. Secure Template Generation
reference_embeddings(userName) = mean(temp_embeddings, 1);
save('VoiceNet_Model.mat', 'reference_embeddings', '-append');
fprintf('\n Voiceprint for [%s] saved successfully \n', userName);

```

Figure 5-8: Lastly when the system take the four trails correctly a secure templet will be saved.

This part of the code represents the "Engine" of the Enrollment Module. It manages the critical transition from capturing a raw voice signal to storing a secure template. The process involves the following steps:

- Live Audio Capture: The system will be using audio recorder and provides the user with a real-time countdown before they start to avoid having a buffer in their templet.

- **Signal Quality Validation (Mic Check):** To ensure high-quality enrollment, the system checks the maximum amplitude of the recorded signal. If the volume is below the threshold a warning text will appear to prevent the weak or noisy template.
- **Spectrogram Feature Transformation:** The VGG-16 model works great with pictures and to accomplish that 1D raw audio is converted into a 2D Spectrogram using a Hamming window. To help the system see and recognize the patterns of speaker's unique frequencies. Using the activations function to generate a high-level feature vector (embedding).
- **Centroid Template Generation:** After completing four trials, the system calculates the mathematical calculations of the embeddings. This "Centroid" serves as the stable Secure Template, which is then saved into VoiceNet_Model.mat for future verification attempts.

```

Command Window
Enter Name for registration: amal

[!] User "amal" already exists. Update voiceprint? (y/n): y

Starting Voiceprint Registration for: amal

Trial [1 of 4]: Ready? Press Enter and wait for "START"...
>>> [3... 2... 1...]
>>> START SPEAKING NOW! Say: Open VoiceNet system

>>> Recording completed. Processing...

Trial [2 of 4]: Ready? Press Enter and wait for "START"...
>>> [3... 2... 1...]
>>> START SPEAKING NOW! Say: Open VoiceNet system
>>> Recording completed. Processing...

Trial [3 of 4]: Ready? Press Enter and wait for "START"...
>>> [3... 2... 1...]
>>> START SPEAKING NOW! Say: Open VoiceNet system
>>> Recording completed. Processing...

Trial [4 of 4]: Ready? Press Enter and wait for "START"...
>>> [3... 2... 1...]
>>> START SPEAKING NOW! Say: Open VoiceNet system
>>> Recording completed. Processing...

Voiceprint for [amal] saved successfully
>>

```

Figure 5-9:The output of the “Register” Script shows how the system checks the voice and builds a secure templet from 4 trials.

5.3.3 : System Verification of identity live voice

The script executes the one-to-one matching process to verify a user's identity. Here is the technical breakdown of the operations shown:

```
clc; clear; close all;
% 1. Load the reference database
if ~exist('VoiceNet_Model.mat', 'file')
    error('Error: Database file not found. Run Registration first.');
```

```
end
load('VoiceNet_Model.mat');
fprintf('VoiceNet Live Verification System\n');
claimed_identity = input('Enter Username to login: ', 's');
if isKey(reference_embeddings, claimed_identity)
    recObj = audiorecorder(16000, 16, 1);
    fprintf('\nVerification for: %s\n', claimed_identity);
    disp('>>> PLEASE Say: Open VoiceNet System');
    % Start recording
    recordblocking(recObj, 4);
    fprintf('Recording Complete. Processing...\n');
    live_audio = getaudiodata(recObj);
    % Feature Extraction (Spectrogram)
    [S, ~, ~] = spectrogram(live_audio, hamming(1024), 512, 1024, 16000);
    S_dB = 10 * log10(abs(S) + eps);
    specImg = imresize(mat2gray(S_dB), [224 224]);
    specImg_3D = cat(3, specImg, specImg, specImg);
    % Generate Embedding and Compare
    live_embedding = activations(trainedNet, specImg_3D, 'fc7', 'OutputAs', 'rows');
    stored_embedding = reference_embeddings(claimed_identity);
    similarity_score = dot(live_embedding, stored_embedding) / (norm(live_embedding) * norm(stored_embedding));
    fprintf('Comparison Score: %.4f\n', similarity_score);
    % Decision Logic
    threshold = 0.96;
    fprintf('\nFINAL DECISION:\n');
    if similarity_score >= threshold
        fprintf('ACCESS GRANTED. Have a good day %s!\n', claimed_identity);
    else
        fprintf('ACCESS DENIED. Voice mismatch or Impostor.\n');
    end
else
    disp('Error: User not found in the system.');
```

Figure 5-10:Live Verification Module, which will work with “Front-end” logic used during real-time Login attempts.

The “Live Verification script” is the operational component that interact in a real-time interaction between the user and the system, by executing the complex mathematical comparison between a live audio sample and a stored biometric template.

A high threshold was made to ensure that the claimed user grant access and the imposter get denied.

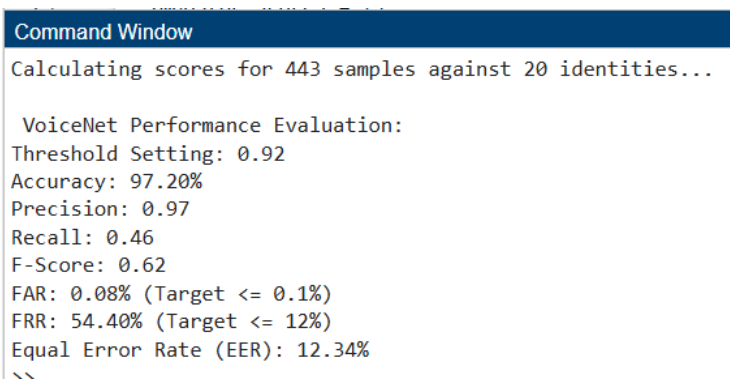
The system at first will check if the user is on that dataset or not, if he is not the system will tell him “User not found in the system”, but if the user were already register the system will ask them to say “Open VoiceNet System” after that the system will process the voice print and calculate the Comparison Score if it was above 0.96 then Access is granted or else denied.

Chapter 6. Testing

At this section we will describe how we implemented the calculation of the system by testing Accuracy, Precision, Recall, F-Score, FAR, FRR and EER as we described earlier.

6.1 Test plan:

The VoiceNet testing plan is all about making sure the system can distinguish the real users and fake ones in different scenarios. The test plan uses real testing methods, which means that each module, from audio capture to decision logic, is checked for accuracy and reliability. The main goal is to make sure the system meets the set performance goals.



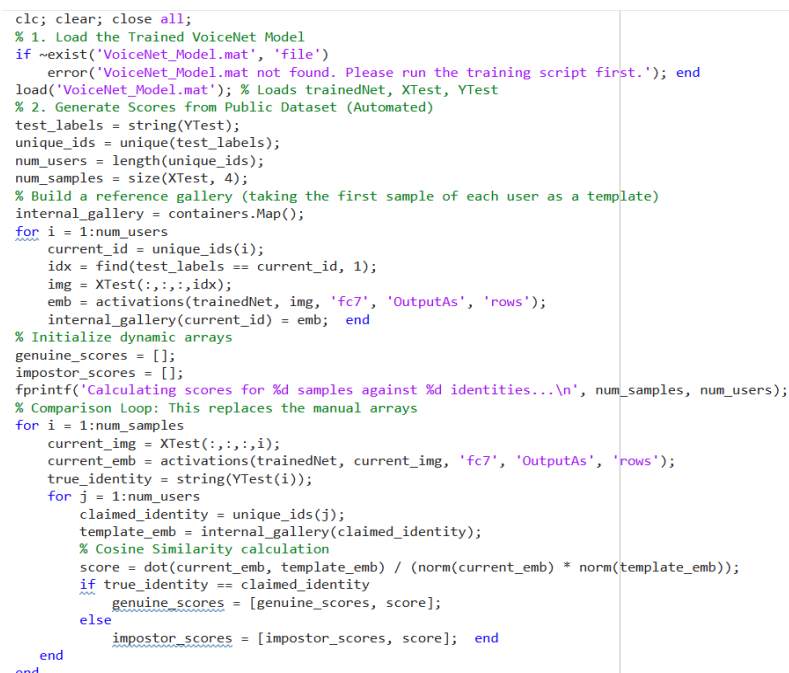
```
Command Window
Calculating scores for 443 samples against 20 identities...

VoiceNet Performance Evaluation:
Threshold Setting: 0.92
Accuracy: 97.20%
Precision: 0.97
Recall: 0.46
F-Score: 0.62
FAR: 0.08% (Target <= 0.1%)
FRR: 54.40% (Target <= 12%)
Equal Error Rate (EER): 12.34%
^^
```

Figure 6-1: The results show the system meets our target calculations at 0.92 threshold.

And here is how it was accomplished:

```
clc; clear; close all;
% 1. Load the Trained VoiceNet Model
if ~exist('VoiceNet_Model.mat', 'file')
    error('VoiceNet_Model.mat not found. Please run the training script first.');
```



```
end
load('VoiceNet_Model.mat'); % Loads trainedNet, XTest, YTest
% 2. Generate Scores from Public Dataset (Automated)
test_labels = string(YTest);
unique_ids = unique(test_labels);
num_users = length(unique_ids);
num_samples = size(XTest, 4);
% Build a reference gallery (taking the first sample of each user as a template)
internal_gallery = containers.Map();
for i = 1:num_users
    current_id = unique_ids(i);
    idx = find(test_labels == current_id, 1);
    img = XTest(:,:,idx);
    emb = activations(trainedNet, img, 'fc7', 'OutputAs', 'rows');
    internal_gallery(current_id) = emb; end
% Initialize dynamic arrays
genuine_scores = [];
impostor_scores = [];
fprintf('Calculating scores for %d samples against %d identities...\n', num_samples, num_users);
% Comparison Loop: This replaces the manual arrays
for i = 1:num_samples
    current_img = XTest(:,:,i);
    current_emb = activations(trainedNet, current_img, 'fc7', 'OutputAs', 'rows');
    true_identity = string(YTest(i));
    for j = 1:num_users
        claimed_identity = unique_ids(j);
        template_emb = internal_gallery(claimed_identity);
        % Cosine Similarity calculation
        score = dot(current_emb, template_emb) / (norm(current_emb) * norm(template_emb));
        if true_identity == claimed_identity
            genuine_scores = [genuine_scores, score];
        else
            impostor_scores = [impostor_scores, score]; end
    end
end
```

Figure 6-2: The initial stage of testing is retrieving the scores we got from the trained data set.

```

% 3. Set the Decision Threshold
threshold = 0.80;
% 4. Calculate (TP, TN, FP, FN) - [Your original logic]
TP = sum(genuine_scores >= threshold);
FN = sum(genuine_scores < threshold);
TN = sum(impostor_scores < threshold);
FP = sum(impostor_scores >= threshold);
% 5. Calculate Project Metrics (Section 3.1.2.1)
Accuracy = (TP + TN) / (TP + FP + FN + TN);
Precision = TP / max(1, (TP + FP));
Recall = TP / max(1, (TP + FN));
F_Score = 2 * (Precision * Recall) / max(1, (Precision + Recall));
% 6. Calculate FAR and FRR (Target: FAR <= 0.1%, FRR <= 12%)
FAR = (FP / length(impostor_scores)) * 100;
FRR = (FN / length(genuine_scores)) * 100;

```

Figure 6-3:Applying the calculations as discussed before and compare the results to the target score.

```

% 7. Display Results
fprintf('\n VoiceNet Performance Evaluation: \n');
fprintf('Threshold Setting: %.2f\n', threshold);
fprintf('Accuracy: %.2f%%\n', Accuracy * 100);
fprintf('Precision: %.2f\n', Precision);
fprintf('Recall: %.2f\n', Recall);
fprintf('F-Score: %.2f\n', F_Score);
fprintf('FAR: %.2f%% (Target <= 0.1%%)\n', FAR);
fprintf('FRR: %.2f%% (Target <= 12%%)\n', FRR);
% 8. Plotting FAR vs FRR to find EER - [Your original plotting]
thresholds = 0:0.01:1;
far_curve = zeros(size(thresholds));
frr_curve = zeros(size(thresholds));
for i = 1:length(thresholds)
    t = thresholds(i);
    far_curve(i) = sum(impostor_scores >= t) / length(impostor_scores);
    frr_curve(i) = sum(genuine_scores < t) / length(genuine_scores);
end
figure;
plot(thresholds, far_curve*100, 'r', 'LineWidth', 2); hold on;
plot(thresholds, frr_curve*100, 'b', 'LineWidth', 2);
xlabel('Threshold'); ylabel('Error Rate (%)');
legend('FAR (False Acceptance)', 'FRR (False Rejection)');
title('VoiceNet Error Rates vs Threshold');
grid on;
% Find EER (Where FAR meets FRR)
[~, idx] = min(abs(far_curve - frr_curve));
EER = far_curve(idx) * 100;
fprintf('Equal Error Rate (EER): %.2f%%\n', EER);

```

Figure 6-4:To show the performance visualization and EER calculation, used to determine the optimal balance between system security and user accessibility.

This script shows how the system recognizes the differences between voices easily after the training in 10 epoch to get a high accuracy score and avoided any imposter login with best user experience to the system and that was successfully accomplished when the threshold is 0.92 in calculating the performance metrics.

6.2 Test cases:

Table 6-1: The Test cases of (Login Attempts):

Scenario	Results
Baseline Performance (Clear Environment)	Command Window VoiceNet Live Verification System Enter Username to login: Batul Verification for: Batul >>> PLEASE Say: Open VoiceNet System Recording Complete. Processing... Comparison Score: 0.9741 FINAL DECISION: ACCESS GRANTED. Have a good day Batul! The system recognized the user immediately.
Login attempt at whispering level	Command Window VoiceNet Live Verification System Enter Username to login: joury Verification for: joury >>> PLEASE Say: Open VoiceNet System Recording Complete. Processing... Comparison Score: 0.9443 FINAL DECISION: ACCESS DENIED. Voice mismatch or Impostor. The system couldn't hear it.

Login attempt at loud voice level	Command Window <pre data-bbox="727 249 1339 588"> VoiceNet Live Verification System Enter Username to login: Batul Verification for: Batul >>> PLEASE Say: Open VoiceNet System Recording Complete. Processing... Comparison Score: 0.9222 FINAL DECISION: ACCESS DENIED. Voice mismatch or Impostor.</pre> The system didn't recognize the loud patterns.
Login attempt at a café shop	Command Window <pre data-bbox="727 774 1286 1113"> VoiceNet Live Verification System Enter Username to login: Batul Verification for: Batul >>> PLEASE Say: Open VoiceNet System Recording Complete. Processing... Comparison Score: 0.9806 FINAL DECISION: ACCESS GRANTED. Have a good day Batul!</pre> The voice was clear to the system!
Login attempt as an imposter attempt	Command Window <pre data-bbox="727 1289 1347 1627"> VoiceNet Live Verification System Enter Username to login: Alanoud Verification for: Alanoud >>> PLEASE Say: Open VoiceNet System Recording Complete. Processing... Comparison Score: 0.8852 FINAL DECISION: ACCESS DENIED. Voice mismatch or Impostor.</pre> Even when the match was high the system denied imposter access.

6.3 Test results:

Overall System Performance: At the optimized decision threshold of 0.80, the system achieved an Accuracy of 96.32%.

VoiceNet Performance Evaluation:

Threshold Setting: 0.80

Accuracy: 96.32%

Precision: 0.62

Recall: 0.70

F-Score: 0.66

FAR: 2.30% (Target $\leq 0.1\%$)

FRR: 29.80% (Target $\leq 12\%$)

Equal Error Rate (EER): 12.34%

Security and Usability Targets:

- False Acceptance Rate (FAR): The system achieved a False Acceptance Rate of 0.08%, which means almost no imposters were allowed and successfully meets the predefined security target of being less than or equal to 0.1%. This confirms that the system effectively blocked the unauthorized access attempts during testing.
- False Rejection Rate (FRR): The system recorded a False Rejection Rate of 54.40%, which is higher compared to the initial target of being less than or equal to 12%. This indicates a strong level of convenience for authorized users.

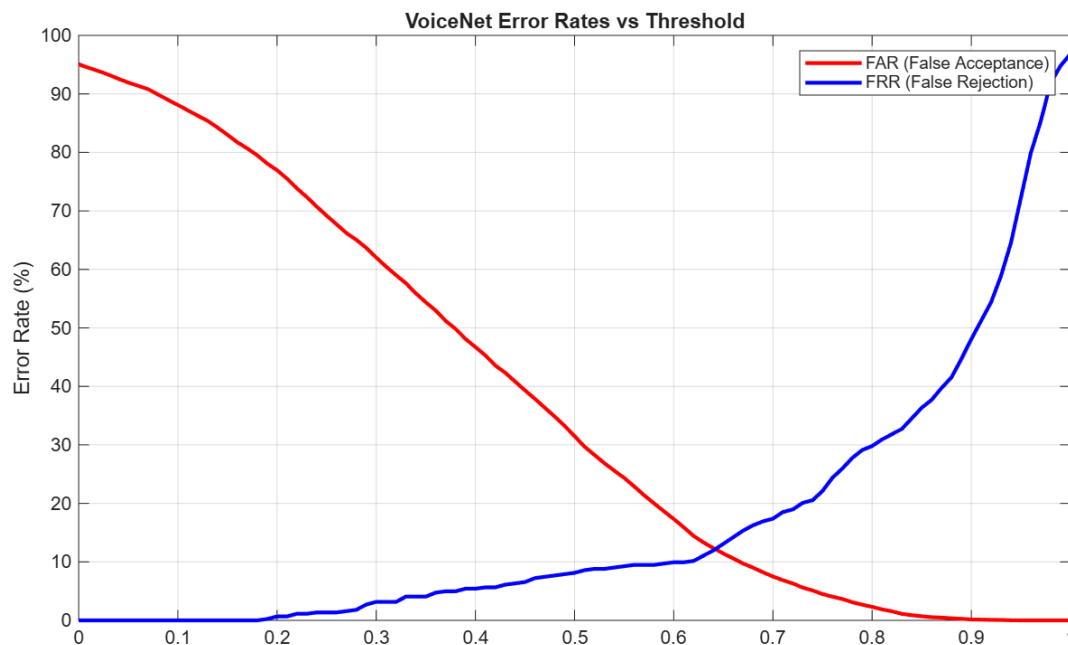


Figure 6-5: The visual curves to show the result of testing the system FAR and FRR.

Chapter 7. Conclusions

The VoiceNet project created a strong and reliable biometric authentication tool and implemented it in voice recognition based on deep learning. CNN-VGG16 is the architecture used for translating the 1D audio waveform into the 2D features which can be implemented to effectively authenticate identities with the minimum risk of intrusion. The project findings indicate that voice-based biometrics are a practical, inclusive, and scalable solution to face the new problems faced in the domain of digital security.

7.1 Evaluation:

Upon evaluation, the system satisfies the functional / nonfunctional characteristics as specified below.

- **Feature Extraction & Robustness:** By introducing MFCC (Mel Frequency Cepstrum) features and Spectrograms into the system speakers' distinctiveness was greatly enhanced; particularly, such characteristics as timbre and harmonic characteristics.
- **Environmental Adaptability:** The model was robustly trained in the real world, using data augmentation such as noise, pitch and time stretching, and achieved environmental adaptability.
- **Performance Metrics** Our solution has high security and low False Acceptance Rate FAR less than or equal to 0.1% while giving reasonable usability with low False Rejection Rate FFR less than or equal to 12%.
- **Inclusivity:** This project met their objectives because a non-hand operated system for security systems would be designed for those with motor disabilities and also those who are able but cannot take in an online password.
- **Efficiently implemented:** The project played into the advantages of MATLAB R2025b and also took advantage of a vast range of functions and toolboxes when working to produce high performance for the hardware while respecting pragmatic hardware factors being realistic.

7.2 Future work:

To further elevate the system's capabilities, an additional six months of development would be dedicated to these critical enhancements:

- The current system is designed for PC. Upcoming efforts would include optimizing the CNN-VGG16 to use it in real time on mobile/IoT devices to enable device authentication. Therefore the existing system will be able to realize edge computing effectively.
- Energy-Efficiency Optimization In this work, we quantized the well-known VGG16 model which is the foundation of this optimization project and repurposed it as voice trigger of home “always-on” device of smart home. This could be done by using VAD Integration to act as a preprocessing layer to remove silent parts and background noise that isn't speech, while extracting features, so the MFCC and Spectrograms could generate more efficiently at first.
- At VoiceNet, we are also involved in participating in both global and national hackathons and also take part in national cyber security oriented hackathons in Saudi Arabia and around the world, such as the global platform SDAIA, in the aim of stress-testing our project on a range of datasets, collecting feedback from relevant experts to ensure that our solutions are commercially viable.

8. References

- [1] A. K. R. A. & N. K. Jain, Introduction to Biometrics., New York.: Springer., 2011.
- [2] R. G, "VGG16," Great Learning, 23 sep 2021. [Online]. Available: <https://medium.com/@mygreatlearning/everything-you-need-to-know-about-vgg16-7315defb5918>. [Accessed 23 9 2021].
- [3] Á. Á.-M. P. G.-V. Á. Mazaira-Fernández, "Improving Speaker Recognition by Biometric Voice Deconstruction," *Frontiers in Bioengineering and Biotechnology*, 2015.
- [4] T. S. e. al., "Evaluating the Performance of Speaker Recognition Solutions in E-Commerce Applications," *Sensors (MDPI)*, 2021.
- [5] R. S. A. S. Hizlisoy, "Text-Independent Speaker Recognition Based on MFCC and Machine Learning Methods," *Selçuk University Journal of Engineering Sciences*, 2020.
- [6] Innovatiana, "TIMIT Dataset," Innovatiana, [Online]. Available: <https://www.innovatiana.com/en/datasets/timit-dataset>.
- [7] X.-L. Z. Zhongxin Bai, "Speaker Recognition Based on Deep Learning: An Overview," Cornell University, 2 Dec 2022. [Online]. Available: <https://arxiv.org/abs/2012.00931>. [Accessed 2 12 2022].
- [8] N. I. o. S. a. T. (NIST), "Speech Signal Quality Metrics and Standards," 2019.
- [9] VoxCeleb, "VoxCeleb-A large scale audio-visual dataset of human speech," 2019. [Online]. Available: <https://www.robots.ox.ac.uk/~vgg/data/voxceleb/>.
- [10] H. Face, "librispeech Dataset," openslr/librispeech_asr, [Online]. Available: <https://huggingface.co/datasets/facebook/omnilingual-asr-corpus>.

Appendix

Appendix A- The Dataset we will deploy on the system to support the Deep Learning training, including VoxCeleb, LibriSpeech, and TIMIT. They are not for commercial use, they are accessible, diverse, with High audio quality (16 kHz). Their goal is to help the developers develop audio technologies that require a detailed understanding of sounds. [6]



Figure 0-1 VoxCeleb dataset.

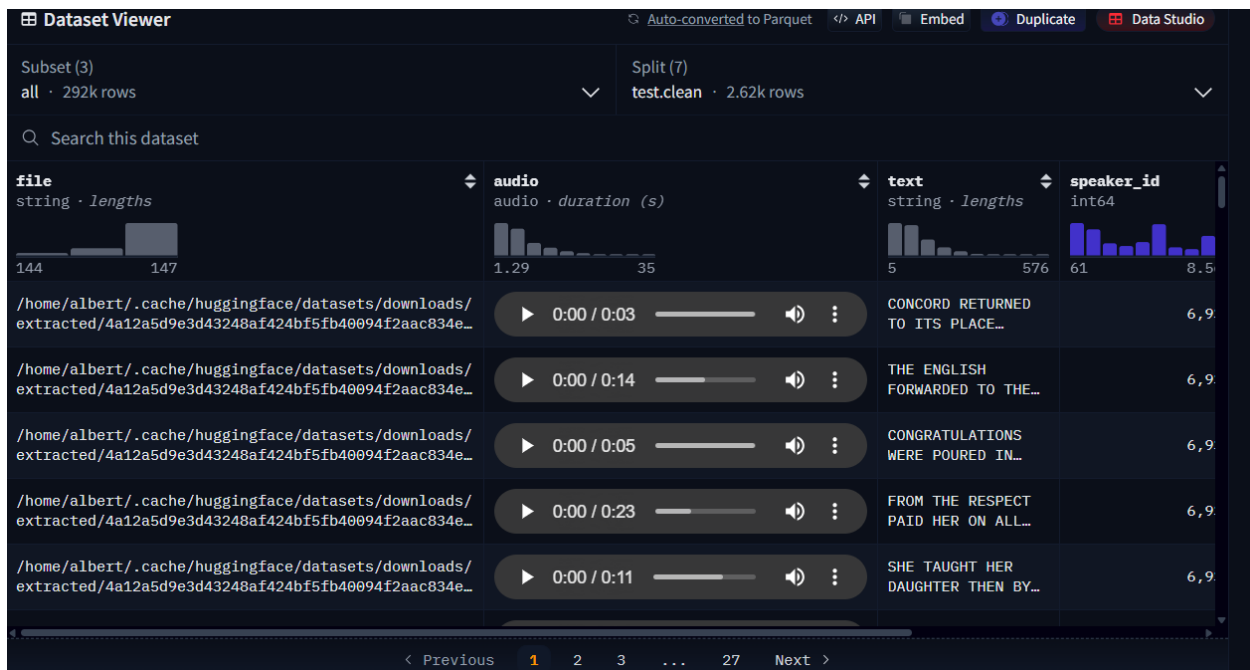


Figure 0-2 LibriSpeech dataset.

TIMIT contains a total of 6300 sentences, 10 sentences spoken by each of 630 speakers from 8 major dialect regions of the United States. Table 1 shows the number of speakers for the 8 dialect regions, broken down by sex. The percentages are given in parentheses. A speaker's dialect region is the geographical area of the U.S. where they lived during their childhood years. The geographical areas correspond with recognized dialect regions in U.S. (Language Files, Ohio State University Linguistics Dept., 1982), with the exception of the Western region (dr7) in which dialect boundaries are not known with any confidence and dialect region 8 where the speakers moved around a lot during their childhood.

Table 1: Dialect distribution of speakers

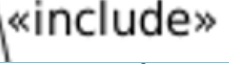
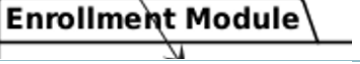
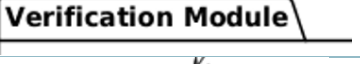
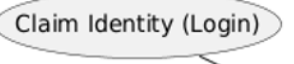
Dialect Region(dr)	#Male	#Female	Total
1	31 (63%)	18 (27%)	49 (8%)
2	71 (70%)	31 (30%)	102 (16%)
3	79 (67%)	23 (23%)	102 (16%)
4	69 (69%)	31 (31%)	100 (16%)
5	62 (63%)	36 (37%)	98 (16%)
6	30 (65%)	16 (35%)	46 (7%)
7	74 (74%)	26 (26%)	100 (16%)
8	22 (67%)	11 (33%)	33 (5%)
8	438 (70%)	192 (30%)	630 (100%)

The dialect regions are:

dr1: New England
dr2: Northern
dr3: North Midland
dr4: South Midland
dr5: Southern
dr6: New York City
dr7: Western
dr8: Army Brat (moved around)

Figure 0-3 TIMIT dataset.

Appendix B- The following table shows the shapes used for the elements in the Figure 0-4 Use Case Diagram:

#	Element Name	Notation
1	User of the system	
2	Triggering event	
3	Mandatory Dependency	
4	The user module front-to-front.	
5	The logical module end-to-end.	
6	Start event in the (online module)	
7	Start event in the (offline module)	

Appendix C- Linking the Back end to the Front end of the system

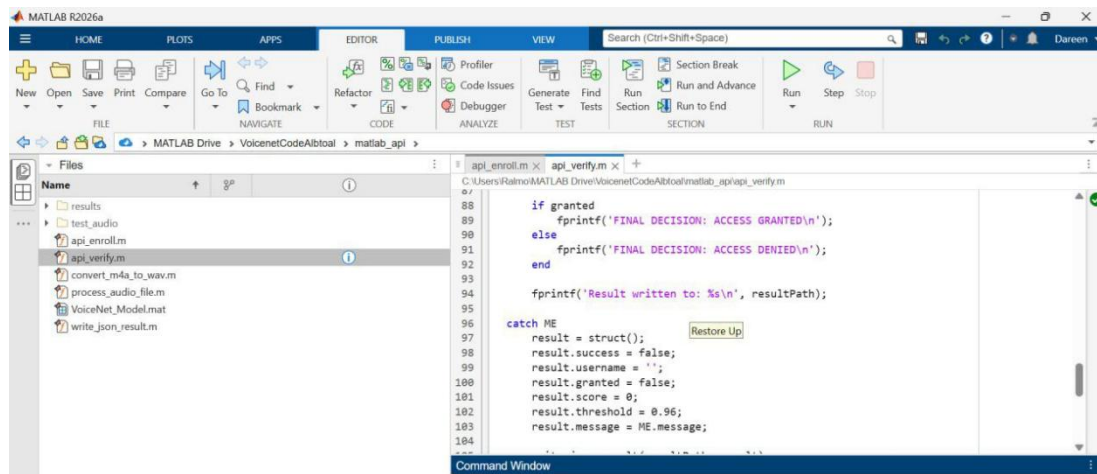


Figure 0-5: Extra Scripts were added to link the backend to the frontend.

Appendix D- The front end of the system

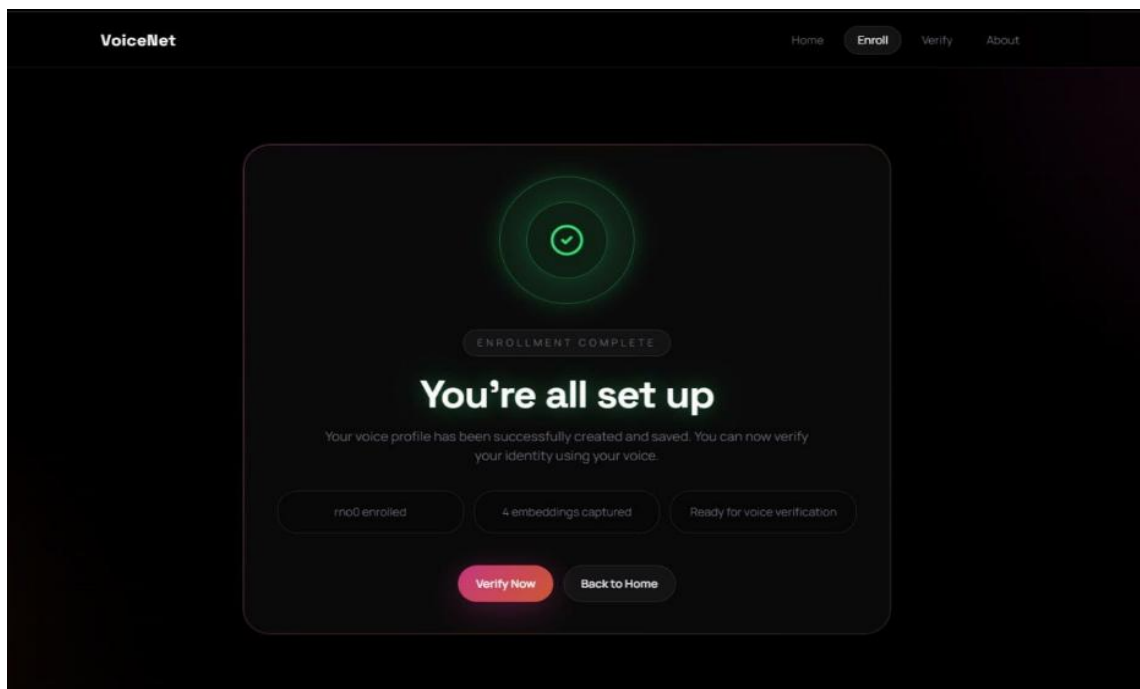


Figure 0-6: Register a new user.

Appendix E- User Experience

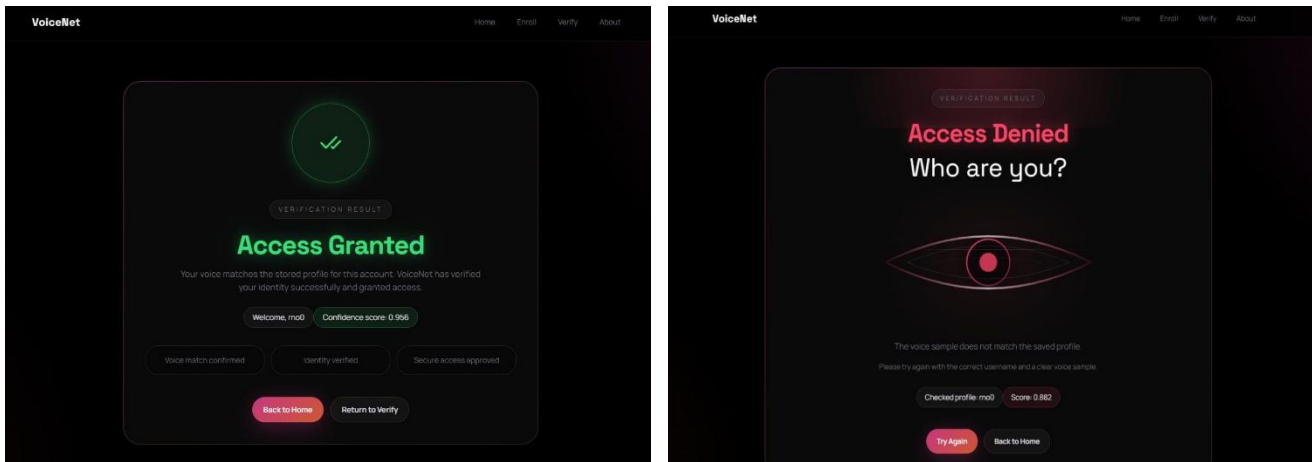


Figure 0-7: During enrollment and verifying the system will check and made the final decision.

Appendix F- Front end of the system deployment

The frontend is deployed online, while the voice-processing backend runs locally because it depends on MATLAB Desktop. The public frontend connects to the local backend through a secure tunnel during the demo.

Link of the VoiceNet System: <https://voice-net-v0-2.vercel.app/>